Scout Before You Route: Predicting Task Success and Token Count from a Small LLM Prefill

Aristides Milios *aristides.milios@mila.quebec*
Mila – Quebec AI Institute, Université de Montréal
ServiceNow AI Research

Christopher Pal *christopher.pal@mila.quebec*
Mila – Quebec AI Institute, Polytechnique Montréal
ServiceNow AI Research
Canada CIFAR AI Chair

Abstract

Routers typically choose a model per query by trading predicted correctness against cost, but most price each model at a fixed cost, as if every query drew the same number of output tokens. The output length of reasoning models varies by an order of magnitude across queries. When is per-query cost worth predicting? What factors drive cost? How cheaply can predictions be made? We predict **both** the success probability **and** the output length of candidate models from a single small frozen LLM *scout's* prefill activations. Perfect per-query cost knowledge is worth 29–61% of spending with reasoning candidates, and much less with non-reasoning ones on the same problems. Across nine datasets, our predicted costs reduce spending at matched accuracy by 9–33% relative to median-length pricing on eight, up to two thirds of that headroom, and no alternative estimator we evaluated is significantly cheaper on any dataset, including reimplementations of ZeroRouter and MixLLM and a similarly sized text-embedding model. Output length depends mostly on the problem: where it tracks difficulty, pricing from predicted difficulty is nearly enough, and where it does not, only a dedicated length probe captures it among the estimators we tried. The probe keeps most of its saving with 20 labelled problems, transfers without labels to four of seven new benchmarks, and on a mixture of all nine benchmarks saves 22% beyond pricing each benchmark at its own median length. Savings saturate by a 2B prefill and do not grow with size, so above 4B a larger prefill costs more than it adds.

1 Introduction

Routing reasoning models requires predicting both correctness and cost before generation. A model's price per token is known in advance, but the number of tokens it will write for a given problem is not. A model that is cheap on average can be expensive on a particular problem: on LiveCodeBench, a single model's output length differs 11–121× between its 10th- and 90th-percentile problems (Fig. 3).

Prefill routers predict correctness from a small encoder's prompt activations; the prefill router of Varshney et al. (Varshney et al., 2026) explicitly prices outputs at median training length. We read output length from the same activations of a small frozen *scout* model, which only reads the prompt and generates nothing (Fig. 2). The scout is in effect an *actuarial scout*: as an actuary prices each policy from the applicant rather than charging everyone the average premium, it prices each query before any model is called. Cost prediction adds one linear probe per candidate to an existing prefill router, without another encoder pass. Most routers that do predict per-query cost evaluate it only inside their full system; the one that isolates it finds that it adds little (Somerstep et al., 2025), on pools with almost no reasoning models. We isolate it on reasoning candidates, against both constant pricing and perfect per-query cost knowledge.

This raises three questions, which organise the paper:

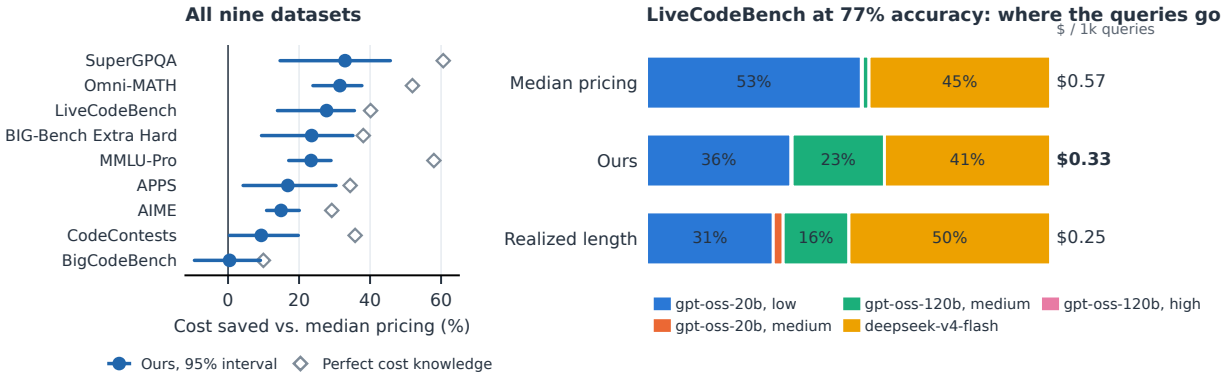


Figure 1: **What predicting cost saves, and where the queries go.** Left: cost saved relative to median pricing on all nine datasets, with 95% intervals, and the saving of perfect per-query cost knowledge (Table 3). Right: share of LiveCodeBench test queries sent to each candidate at 77% accuracy by three routers that price a query by median length, by our cost probes, or by its realized length (perfect knowledge), with the spend per 1,000 queries (one training set). Ours moves queries from gpt-oss-20b at low effort to gpt-oss-120b at medium effort.

1. When is per-query cost worth predicting?
2. What factors drive cost for a given input?
3. How cheaply can a router obtain a cost prediction: with how many labels, and with a prefill pass from how small an LLM?

We contribute:

1. a measurement of what perfect knowledge of per-query cost saves: 29–61% of spending at matched accuracy on eight of nine datasets with reasoning candidates, and much less with non-reasoning candidates on the same problems (Section 4);
2. a prefill cost probe that saves 9.4–33.0% relative to median pricing on eight of nine datasets, up to two thirds of the perfect-knowledge headroom, and that no alternative cost estimator we tried significantly beats on any dataset (Fig. 1 and Table 1);
3. a mechanism: length depends mostly on the problem (model lengths co-vary, and models output relatively consistent lengths across rollouts on the same query). Where length correlates with difficulty, length from predicted success suffices, and where it does not (MMLU-Pro, BIG-Bench Extra Hard), only the cost probe, among the estimators we tried, captures it. Our two failures come from too little headroom, or headroom no estimator we tried can capture (Sections 6 and 7);
4. generalisation: the cost probe keeps most of its saving with 20 labelled problems and for a new model onboarded from 10 examples; it transfers with no target labels to four of seven benchmarks; and in mixed traffic it saves 22.4% beyond pricing by benchmark identity (Section 8);
5. prefill size and controls: savings do not grow with the prefill from 1.7B to 14B, so above 4B a larger prefill costs more than it adds, while a text-embedding model of the same size and fully fine-tuned small models read cost worse (Sections 8 and 9).

Related work. Learned routers pick one model per query from the prompt, trading predicted quality against price: RouteLLM learns a strong-versus-weak decision from preference data (Ong et al., 2024), RouterBench standardizes the evaluation (Hu et al., 2024), and IRT-Router models query difficulty and model ability with item response theory (Song et al., 2025). Closest to us, the prefill router of Varshney et

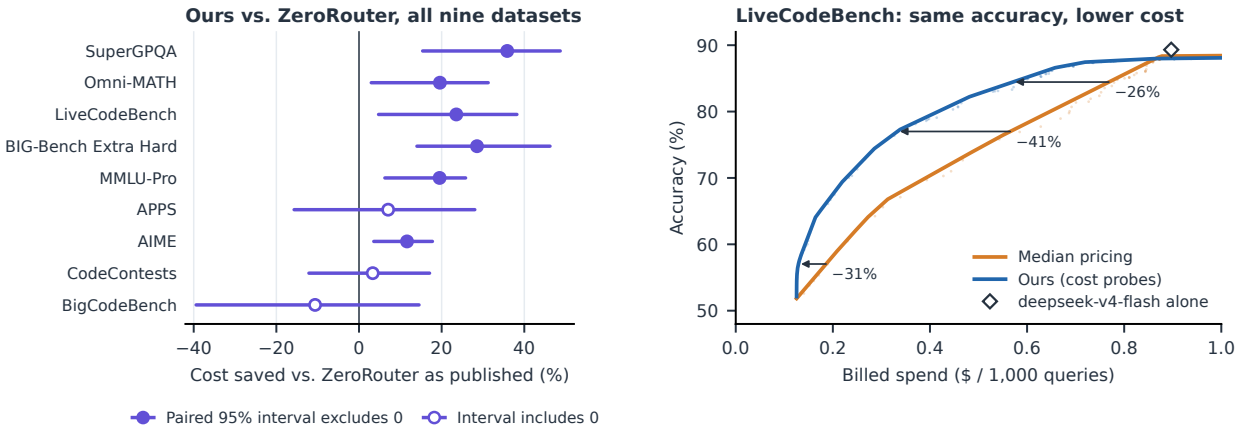


Figure 2: **Cost saved at matched accuracy, billed prices.** Left: cost saved by ours relative to ZeroRouter as published (its own success model and pricing), with 95% intervals; hollow where the interval includes zero (Tables 1 and 9). Right: LiveCodeBench cost–accuracy frontiers of our cost probe and median pricing (one training set); arrows mark the saving at matched accuracy, and the diamond is deepseek-v4-flash on every query.

al. predicts each model's correctness from a small encoder's prompt activations (Varshney et al., 2026), and pre-generation activations are known to encode whether a model will fail (Lugoloobi et al., 2026). These routers, like SWE-Router for agentic software engineering (Son et al., 2026), price each model at a per-model constant; the prefill router uses the median training output length, which is our main reference. Fewer routers predict per-query cost: MixLLM regresses output length on query embeddings (Wang et al., 2025), CARROT uses embedding nearest neighbours or a fine-tuned RoBERTa (Somerstep et al., 2025), GraphRouter predicts cost on a query–model graph (Feng et al., 2024), Route-To-Reason predicts reasoning-model length from text embeddings (Pan et al., 2025), ZeroRouter prices queries by item-response difficulty bins (Yan et al., 2026), and SCOPE estimates a query's cost from retrieved past behaviour (Cao et al., 2026). R2-Router routes over pairs of a model and an output-length budget, so length is controlled rather than predicted (Xue et al., 2026), and SCX Router reads task type, difficulty and an expected output length from a small decoder's KV cache as properties of the task, not of each candidate model (Stepanov et al., 2026). Of MixLLM, CARROT and Route-To-Reason, only CARROT isolates its cost predictor, against the same router pricing each model at its training-average cost, and it finds that per-query cost adds little on RouterBench and SPROUT (Somerstep et al., 2025); none of the three compares with perfect per-query cost knowledge. We compare cost estimators directly, including reimplementations of these and ZeroRouter, with every estimator given the same success predictions and measured against both constant pricing and that oracle. Recent large-scale evaluations find that routers land close to an oracle that assigns each dataset its best model, and attribute much of their gain to coarse domain structure (Li et al., 2026a); a static assignment by task type closes most of the gap to the per-query oracle on a small benchmark (Lee, 2026). Both concern which model answers correctly; we measure what per-query cost is worth beyond the benchmark a query comes from.

Cascades instead call models in sequence and stop when an answer is accepted: FrugalGPT learns a scorer that decides whether to escalate (Chen et al., 2023), AutoMix uses self-verification (Aggarwal et al., 2024), and Dekoninck et al. unify routing and cascading (Dekoninck et al., 2024). A cascade pays for the cheap attempt on every query; we route once, before generation. Without a verifier, a pre-generation router beats the best cascade on four of five datasets, LiveCodeBench among them (Bouchard, 2026). Activation-based length prediction also supports serving and scheduling of a single model (Messenger, 2026; Xie et al., 2026; Wen et al., 2026); we predict the length of other models from one shared prefill. Hosted open-weight models vary across providers in accuracy, length and price (Li et al., 2026b; Lin et al., 2026; Gao et al., 2025), and FACET selects among providers of a fixed model per task from measured outcomes (He et al., 2026).

2 Cost Prediction and Routing

For query x and candidate m (a model at a fixed reasoning effort, served by an endpoint), let $p_m(x)$ be the probability of a correct answer and $c_m(x)$ the expected generation cost. One candidate is chosen before generation,

$$m^*(x; V) = \arg\max_m \{V\hat{p}_m(x) - \hat{c}_m(x)\}, \tag{1}$$

where V is the value of a correct answer; sweeping V trades accuracy for cost. Every query is answered once, with no verifier or fallback.

Probes. We concatenate mean- and last-token activations from eight layers of a frozen Qwen3-4B (Yang et al., 2025a) and fit one linear probe (Alain & Bengio, 2016) per candidate and target: for success an L2-regularized logistic regression, calibrated by Platt scaling (Platt, 1999); for cost a ridge regression of log mean output length, corrected for the bias of exponentiating by Duan's smearing (Duan, 1983). The predicted length is priced at the candidate's token rates, $\hat{c}_m(x) = r_m^{\text{in}} n_m^{\text{in}}(x) + r_m^{\text{out}} \hat{\ell}_m(x)$. Appendix B gives the features, regularization and calibration.

3 Experimental Protocol

The main pool has five candidates, called through OpenRouter: gpt-oss-20b at low and medium effort (OpenAI, 2025), deepseek-v4-flash (DeepSeek-AI, 2026), and gpt-oss-120b at medium and high effort. Endpoints of the same open-weight model differ across providers in accuracy, length and price (Li et al., 2026b; Lin et al., 2026), so we pin deepseek-v4-flash, whose output length depends on the provider, to one provider; the gpt-oss candidates are served by whichever provider OpenRouter selects. We use nine datasets: LiveCodeBench (Jain et al., 2024), Omni-MATH (Gao et al., 2024), MMLU-Pro (Wang et al., 2024), AIME (di-zhang-fdu, 2024), APPS (Hendrycks et al., 2021), BigCodeBench (Zhuo et al., 2025), CodeContests (Li et al., 2022), SuperGPQA (M-A-P Team, 2025) and BIG-Bench Extra Hard (Kazemi et al., 2025). Each is split into training, calibration and test problems (Appendices A and B). On three of the datasets we also extend the pool with three reasoning models from other families (Appendix G). Table 2 gives each candidate's accuracy and cost on every test set.

Prices. Each model has one price per input and per output token: its average billed price, a least-squares fit of the billed cost of our calls on their tokens (USD per million output tokens: 0.13 for gpt-oss-20b, 0.08 for deepseek-v4-flash, 0.26 for gpt-oss-120b; for the gpt-oss models, an average over the providers OpenRouter chose). A call's cost is its tokens at these prices; on the Omni-MATH and MMLU-Pro test sets, where we recorded each call's billed cost, we use that instead. Costing those two test sets as tokens, or fitting the prices on training calls only, moves our saving relative to median pricing by at most 1.0 point and changes the sign or significance of no comparison in Table 9 (Appendix B).

Evaluation. Every estimator is fitted on training problems only and evaluated on the test set, with the same success predictions. Sweeping V gives one policy per value and a (cost, accuracy) point on the test set; an estimator's frontier is the upper concave hull of its points, each of which is achievable by sending each query to one of two policies at random with fixed probabilities (Hu et al., 2024). We compare two cost estimators a, b at 12 accuracies that both frontiers reach and report $1 - \exp(\overline{\log C_a/C_b})$, the cost saved by a at matched accuracy. An oracle that prices each problem at its realized output length bounds every estimator; its saving over median pricing is the *headroom*, and our share of it the *capture*. Every result is a mean over ten *training sets*, each of which reshuffles the training and calibration problems, keeps the test set and refits everything; intervals are paired bootstrap intervals over test problems, pooled over the ten. Analyses marked "one training set" use one split (Appendix B). Section 10 checks the results without interpolation, with V fixed in advance on calibration.

Cost saved by ours vs.	LCB	Omni	MMLU-Pro	AIME	APPS	BCB	CC	SuperGPQA	BBEH
External estimators									
Median pricing	**27.8**	**31.5**	**23.4**	**14.9**	**16.8**	0.4	**9.4**	**33.0**	**23.5**
Mean pricing	**27.8**	**34.3**	**25.8**	**15.0**	**13.3**	3.8	**9.4**	**35.8**	**22.7**
Prompt-feature GBM	**20.7**	**25.8**	**20.0**	**14.2**	9.7	1.4	8.0	**19.1**	2.1
MixLLM-style embeddings	**16.2**	**24.7**	**15.5**	**13.5**	**16.4**	1.7	4.3	**15.7**	−1.3
Route-To-Reason	**17.7**	**23.2**	**14.8**	**13.9**	**13.4**	2.9	5.0	**15.1**	4.5
ZeroRouter (as published, DistilBERT)	**23.5**	**19.6**	**19.5**	**11.6**	7.1	−10.6	3.3	**35.9**	**28.6**
ZeroRouter (4B prefill)	**19.3**	14.2	**18.4**	**8.3**	2.3	−10.7	−1.0	**35.2**	**24.5**
Our ablations									
Length from predicted difficulty (bins)	5.7	5.8	**16.7**	**7.9**	2.2	2.4	1.7	**25.1**	**14.2**
Length from predicted success	3.2	−3.0	**18.4**	4.3	−2.2	2.4	−0.8	15.3	**22.6**
One length per query (SCX-style)	**10.7**	**21.1**	**10.4**	**8.8**	5.7	3.0	0.6	5.7	5.5

Table 1: **Cost saved (%) by our cost probe relative to each estimator** at matched accuracy. ZeroRouter uses its own success model, with item parameters read by its fine-tuned DistilBERT (as published) or from our 4B prefill. **Bold**: statistically significant (95% paired bootstrap interval excludes zero). No estimator is significantly cheaper than ours on any dataset. Estimators are described in Appendix B.5.

	LCB		Omni		MMLU-Pro		AIME		APPS		BCB		CC		SGPQA		BBEH	
Candidate	Acc.	$	Acc.	$	Acc.	$	Acc.	$	Acc.	$	Acc.	$	Acc.	$	Acc.	$	Acc.	$
20b, low	51.8	0.13	41.9	0.21	62.1	0.05	57.2	0.26	58.4	0.12	45.3	0.06	49.1	0.14	31.0	0.05	23.3	0.11
20b, medium	68.2	0.65	57.5	1.20	69.4	0.18	88.3	1.18	73.5	0.74	48.3	0.12	70.6	0.85	36.0	0.24	34.8	1.21
deepseek-v4-flash	89.3	0.90	**71.3**	1.66	**86.2**	0.33	**98.3**	0.86	84.1	1.05	52.1	0.11	84.6	1.15	**67.9**	0.47	**64.7**	1.54
120b, medium	82.1	0.64	60.3	1.12	77.2	0.23	89.5	1.23	79.3	0.72	**53.4**	0.18	78.4	0.84	46.0	0.27	45.3	1.45
120b, high	**89.5**	3.73	66.6	4.57	79.3	0.90	97.0	3.14	**85.7**	4.48	53.0	0.64	**85.9**	4.87	47.8	1.48	54.7	5.26

Table 2: **Each candidate alone, without routing.** Accuracy (Acc., %) and mean cost per call ($, USD per 1,000 calls) when every test problem goes to one candidate (billed costs on Omni-MATH and MMLU-Pro, tokens at billed prices elsewhere). SGPQA: SuperGPQA. Bold: most accurate. Costs differ 11–49×, and accuracy is not monotone in price: deepseek-v4-flash is the most accurate on five datasets and within 1.6 points of gpt-oss-120b at high effort on the rest, at 17–37% of its cost.

4 How much is per-query cost worth?

A cost estimator can be no better than knowing each query's realized cost. Table 3 reports this headroom on all nine datasets, with our saving and the share of it we capture (Section 5).

Headroom is large on eight of nine datasets. On eight of nine datasets perfect per-query cost knowledge would save 29–61%; BigCodeBench has little to capture (10.0%; Section 6).

Per-query cost is worth less with non-reasoning candidates. We also collected five non-reasoning candidates on the same problems (deepseek-v4-flash with thinking disabled, Llama-3.1-8B (Grattafiori et al., 2024), Qwen3-30B-A3B-Instruct, Llama-3.3-70B and Qwen3-235B-A22B-Instruct) and fitted the probes as before (Appendix D). Headroom is 1.3–2.3× smaller than with reasoning candidates, our saving 2.3× (LCB) to about 30× (MMLU-Pro) smaller (Table 10), and no other estimator does better on Omni-MATH or MMLU-Pro. Much of the non-reasoning variation is degenerate output that the prompt predicts poorly, such as Llama-3.1-8B looping to its 16k cap on 9–12% of math problems. With thinking on, deepseek-v4-flash gains 2.8–22.1 points of accuracy for 11–33× the output, and the problems that are long with thinking are long without it (log-length correlation .66–.76).

5 How much of it the cost probe captures

Against median pricing. At matched accuracy, our router spends significantly less than median pricing on eight of the nine datasets, from 9.4% on CodeContests to 33.0% on SuperGPQA (Fig. 1 and Table 3),

Dataset	Domain	Test n	Headroom	Ours vs. median	Capture	Ours vs. length from predicted success
LiveCodeBench	code	341	40.1	27.8 [13.9, 35.6]	.69	3.2
Omni-MATH	math	1,000	51.9	31.5 [23.9, 37.7]	.61	−3.0
MMLU-Pro	knowledge	6,500	58.0	23.4 [17.1, 29.0]	.40	**18.4**
AIME 1983–2024	math	281	29.2	14.9 [10.8, 20.1]	.51	4.3
APPS	code	377	34.4	16.8 [4.2, 30.4]	.49	−2.2
BigCodeBench	code	421	10.0	0.4 [−9.5, 9.1]	.04	2.4
CodeContests	code	266	35.8	9.4 [0.4, 19.8]	.26	−0.8
SuperGPQA	knowledge	300	60.6	33.0 [14.6, 45.7]	.54	15.3
BIG-Bench Extra Hard	reasoning	300	38.1	23.5 [9.4, 35.2]	.62	**22.6**

Table 3: **Headroom and capture on every dataset.** Cost saved (%) relative to median pricing by perfect per-query cost knowledge (headroom) and by our cost probe, with 95% intervals; capture is their ratio. Last column: cost saved by the cost probe relative to our ablation that takes length from predicted success; bold: interval excludes zero.

and on seven of them the cost probe captures two fifths to over two thirds of the headroom (.40–.69). In absolute terms, on LiveCodeBench our router reaches 90% of deepseek-v4-flash's accuracy at 49% of what calling it on every query costs, against 69% for median pricing (geometric mean over ten training sets). The headroom also knows the sampling noise of the scored draws, which no predictor can, so these capture figures are conservative (Appendix E). Section 6 treats BigCodeBench, where the probe saves nothing, and CodeContests.

No estimator beats the cost probe. Against the external estimators (Table 1), ours is ahead in 59 of 63 comparisons, significantly in 44, and significantly behind in none, also on random re-splits that reshuffle the test problems (Table 19). Of the other 19, one is ZeroRouter with our prefill on Omni-MATH (behind ours in 7 of 10 training sets), and the rest fall on four datasets. On BigCodeBench no estimator, not even perfect cost knowledge, is distinguishable from median pricing. On APPS and CodeContests ours is the only estimator significantly cheaper than median pricing, and three to five others lie between the two, distinguishable from neither. On BIG-Bench Extra Hard three text-based estimators save as much as ours, all significantly more than median pricing (Section 7). On LCB, Omni-MATH and MMLU-Pro, every external estimator loses by 14–34 points, with intervals excluding zero except ZeroRouter with our prefill on Omni-MATH (Table 9). Ours is significantly ahead of each in every training set there, except against ZeroRouter: as published, in 9 of 10 sets on LCB and Omni-MATH; with our prefill, in 7 of 10 on Omni-MATH.

Where the saving comes from. The saving is not only a price gap: with every candidate charged the same price per token and realized lengths kept, the cost probe still saves 13.3% (LCB), 14.3% (Omni-MATH) and 36.3% (MMLU-Pro) relative to median pricing (one training set), and price gaps larger than the billed ones shrink both the headroom and the saving (Appendix N). It needs a cheap candidate that solves a good share of problems: without gpt-oss-20b at low effort most of it disappears (LCB 27.8% to 6.5%, Omni-MATH 31.5% to 16.2%; Appendix I). Figure 4 shows which candidates it moves traffic between.

Three further model families. Adding Qwen3-32B, GLM-4.7-Flash and Nemotron-3-Super-120B on LCB, Omni-MATH and a 2,700-problem subset of MMLU-Pro leaves the picture unchanged (Appendix G). Without deepseek-v4-flash, in a pool of both gpt-oss-20b efforts and the three new models, our router still saves 29.5% [17.3, 37.2], 12.4% [−0.2, 21.7] and 22.7% [9.9, 33.6] relative to median pricing: the gain does not rest on one cheap and accurate candidate.

6 Where it fails

Our probes save little on two datasets, for different reasons (Fig. 5). Neither failure could have been forecast from simple statistics: models of headroom fitted on how widely lengths and prices vary do not carry over to held-out datasets (leave-one-dataset-out R^2 at most .25; Appendix F), so headroom has to be measured, which takes each candidate's realized lengths on a labelled sample.

Little headroom: BigCodeBench. BigCodeBench tasks are library-usage functions with short, similar solutions; perfect cost knowledge saves only 10.0% [−1.1, 20.1], and no estimator's saving relative to median pricing excludes zero (ours 0.4 [−9.5, 9.1]).

Headroom no estimator reads: CodeContests. CodeContests has as much headroom as APPS (35.8%), but no estimator captures more than about a quarter of it (ours 9.4% [0.4, 19.8]). Its lengths depend on the problem (between-problem share .92–.96) and shared across candidates (other candidates' lengths explain .67–.88), against our R^2 of .35–.52: length tracks difficulty, but difficulty seems hard to read from these prompts. Oracle difficulty, a problem's solve rate across the other candidates, which requires running and grading every model (Section 7), explains about half of length here, as on LCB, but adding it to the prefill raises R^2 by up to .21, against at most .07 on LCB.

7 What factors drive cost for a given input?

Length depends mostly on the problem. Three measurements on the realized outputs, with no predictor, show this (Appendix F). For every reasoning candidate on every dataset with repeated draws, most of the variance in log length lies between problems rather than between draws of the same problem (.71–.99). Other candidates' lengths on the same problem explain most of a candidate's length (.42–.96, with one exception on BIG-Bench Extra Hard): models agree on which questions are long. The part of length specific to one candidate is small (8–29% of its variance), but it is worth as much for routing as the shared part, or more: an oracle that knows only the candidate-specific part saves about as much relative to median pricing as one that knows only the shared part on LCB (24.8% against 25.5%), and more on Omni-MATH and MMLU-Pro (41.4% and 43.7% against 23.3% and 25.4%; Table 12).

On math and code, length tracks difficulty. To ask whether hard problems are the long ones, we measure a problem's difficulty as its solve rate across the other candidates. This *oracle difficulty* requires running and grading every model on the problem, so no router has it; we use it only to explain length, not as a way to price queries. On LCB, AIME and CodeContests, oracle difficulty alone explains about half of each candidate's log length (median R^2 .45–.57), and on Omni-MATH and APPS about a third (one training set; Appendix E). The prefill already reads most of this: adding oracle difficulty to it raises R^2 by at most .07 on LCB, Omni-MATH and APPS, though by up to about .2 on AIME and CodeContests. This is why length from predicted success comes within a few points of the cost probe on LCB and Omni-MATH (Table 3). Its success logits carry more than difficulty, though: pricing from oracle difficulty alone would save less than pricing from them (Appendix E).

On knowledge and mixed-format datasets, it does not. On MMLU-Pro, SuperGPQA, BIG-Bench Extra Hard and BigCodeBench, oracle difficulty explains less than a fifth of log length. On MMLU-Pro, pricing from oracle difficulty and each question's subject saves under 4% relative to median pricing, against 29% for the cost probe (one training set). Here length from predicted success falls well behind the cost probe: by 18.4% [11.0, 23.5] on MMLU-Pro and by 22.6% [12.3, 31.9] on BIG-Bench Extra Hard, whose 23 tasks differ in format (Tables 1 and 3). We added BIG-Bench Extra Hard and SuperGPQA after a screen for this contrast (Appendix M).

When the text alone suffices. On BIG-Bench Extra Hard the task is visible in the text, so with the full training set text-only estimators read length as well as the prefill: MixLLM-style embeddings save 24.4% relative to median pricing, and ours −1.3% [−5.1, 2.7] relative to them. The prefill still needs no second encoder, and with fewer than 100 labelled problems it leads them on average, though within noise (Appendix B.6). On SuperGPQA, the other dataset of mixed subjects, the text estimators fall behind (ours saves 15.7% [0.1, 27.5] relative to them).

What the probe still misses. Apart from one length per query read from the same prefill, our probe reads length better than every other estimator we tried, on every candidate of every dataset (Table 11). That estimator matches it within .04 on some candidates but saves 8.8–21.1 points less on LCB, Omni-MATH, MMLU-Pro and AIME, because it fixes the ratio of lengths between candidates for every query (Table 1).

Cost estimator, 20 training problems	LCB	Omni-MATH	MMLU-Pro	SuperGPQA	BBEH
Ours (cost probe)	22.6†	22.4†	11.9	18.9	12.2
Prompt-feature GBM	−1.0	−2.3	2.2	8.1	0.9
MixLLM-style embeddings	−2.9	1.9	−0.5	5.0	−5.4
CARROT-style kNN	1.7	3.7	2.3	9.5	3.9
ZeroRouter pricing	1.7	5.4	0.2	−2.7	−1.0
Our ablation: length from predicted success	22.5	26.6	3.2	6.3	−12.3
Ours, all training problems	27.8	31.5	23.4	33.0	23.5

Table 4: **Few labels.** Cost saved (%) relative to median pricing (fitted on the full training set) when each cost estimator is fitted on 20 random training problems, two subsets per training set. †: ours exceeds every external estimator (95% intervals exclude zero). Below the line, for reference: our ablation, which exceeds ours on Omni-MATH, and ours with all training problems.

The probe is still below the ceiling set by the between-problem share. It comes closest on LCB, Omni-MATH and BIG-Bench Extra Hard (on LCB, R^2 .71–.75 against .98–.99) and falls furthest below it on the knowledge datasets, above all for deepseek-v4-flash (on MMLU-Pro, .26 against .92). Differences between candidates are what it reads worst (Appendix F).

8 How cheaply can a router get a cost prediction?

A deployed router has few labels, adds models, meets benchmarks it was not trained on and serves mixed traffic; we test the cost estimators in all four settings (protocols in Appendix B.6) and then ask how small the prefill can be.

Few labels. Table 4 refits every cost estimator on 20 random training problems. Our probe keeps 51–81% of its full-data saving on each of the five datasets, while the external estimators save less than 10%; at 100 problems ours saves 17.9% on MMLU-Pro against at most 6.5% for any of them. With more training problems the external estimators improve but do not catch up: on LCB, Omni-MATH, MMLU-Pro and AIME every one of them still saves significantly less than our probe at the full training set, while ours is within 4.5–6.4 points of its full-data saving at 50 problems. The exception is BIG-Bench Extra Hard, where the task format is visible in the text and the text estimators catch up with more data. Length from predicted success matches ours on LCB and Omni-MATH from 20 problems on, but stays below 7% at every n on MMLU-Pro and BIG-Bench Extra Hard (Fig. 6).

New models. To add a model, a router needs its success and cost probes. We hold out one model at a time, with all of its efforts, and fit its probes from k labelled problems while every other candidate keeps its full probes; ours fits two parameters per probe, an offset to the other candidates' predicted log length and a logistic map from their mean success logit. From 10 examples our router saves 19.2–26.7% on four datasets and 9.7% on BIG-Bench Extra Hard, against 3.4–17.4% for ZeroRouter's onboarding, its main use case (Yan et al., 2026), and leads it on every dataset on average (by 5.2–13.0 points), though at 10 examples no interval excludes zero; at 50 examples ours leads significantly on LCB and SuperGPQA (Table 13). The gain concentrates on deepseek-v4-flash, the one model whose family is not otherwise in the pool.

New benchmarks. We then move a whole router to a benchmark it has not seen, training both probes on the other datasets of the same domain (code, or math and knowledge) with no target labels (Table 5). On LCB and AIME the transferred router keeps 94–96% of its in-domain saving, and on MMLU-Pro and APPS it exceeds it, leading the best transferred external estimator by 7.5–28.9 points (significantly on LCB, MMLU-Pro and AIME). The prefill router's rule, the same transferred success probes with median pricing, saves at most 9.0% on these four and loses money on LCB, so the transferred cost probe carries the result. Transfer is weaker on SuperGPQA, Omni-MATH and BIG-Bench Extra Hard, where a few target labels help. Length from predicted success transfers to fewer benchmarks: the map from difficulty to length differs between benchmarks, while the map from the prefill to length carries over to more of them (Appendix H).

Sources → target	Ours, in-domain	Ours, transferred	Prefill-router rule, transferred	Best external, transferred
APPS, BCB, CC → LCB	27.8	26.0	−9.4	−3.0 (ZeroRouter)
LCB, BCB, CC → APPS	16.8	31.5	9.0	23.7 (ZeroRouter)
Omni, AIME, SGPQA, BBEH → MMLU-Pro	23.4	26.6	1.1	18.1 (kNN)
Omni, MMLU-Pro, SGPQA, BBEH → AIME	14.9	14.3	−1.7	6.8 (kNN)
Omni, MMLU-Pro, AIME, BBEH → SuperGPQA	33.0	22.5	−11.2	17.6 (MixLLM)
Omni, MMLU-Pro, AIME, SGPQA → BBEH	23.5	−3.4	−21.0	−6.5 (ZeroRouter)
MMLU-Pro, AIME, SGPQA, BBEH → Omni-MATH	17.8	4.0	−23.4	1.7 (MixLLM)

Table 5: **A router moved to a new benchmark.** Cost saved (%) relative to the target's own median pricing. Ours: success and cost probes trained on the sources, with every choice made on their calibration splits; external estimators transfer the same way, each with its own success model. Prefill-router rule: the transferred success probes with median pricing. Ours is significantly ahead of every arm in every training set on LCB and MMLU-Pro (9 of 10 on AIME). SGPQA: SuperGPQA; Omni: Omni-MATH's 500-problem set.

	Cost saved relative to		
Router prices each candidate by	one median	median per benchmark	Ours vs. it
Baselines			
One median length (the prefill router's rule)	–	−13.9	31.8 [26.1, 36.7]
Median length per benchmark	12.2	–	22.4 [16.5, 27.6]
Mean length per benchmark	13.5	1.5	21.2 [15.9, 26.0]
MixLLM-style embeddings	18.9	7.6	16.0 [11.1, 19.9]
MixLLM (benchmarks as domains)	19.9	8.8	14.9 [9.9, 20.0]
Our ablations: length from predicted success			
Fitted per benchmark	24.4	13.8	9.9 [5.2, 14.2]
From predicted difficulty (bins), per benchmark	24.1	13.6	10.1 [4.4, 15.1]
Fitted on the mixture	11.7	−0.5	22.8 [17.8, 27.8]
Ours (one probe, benchmark not given)	**31.8**	**22.4**	–
Realized cost (headroom)	46.4	38.9	

Table 6: **One router for mixed traffic.** All nine benchmarks in one mixture (150 test problems each), with the same five candidates and success probes in every row. Cost saved (%) at matched accuracy, with 95% intervals resampled within each benchmark; every "ours vs. it" interval excludes zero in all ten training sets. Per-benchmark arms are told each query's true benchmark (an optimistic baseline); ours is not (Appendix B.5).

Mixed traffic. Much of what quality routers gain can be had by knowing a query's dataset or task type (Li et al., 2026a; Lee, 2026); a cost router might likewise get most of its saving by segmenting traffic. We route one mixture of all nine benchmarks (Hu et al., 2024; Li et al., 2026a) and compare with routers told each query's true benchmark (Table 6), an optimistic baseline, since a deployed router would have to infer it with some error. Pricing each candidate at its median length within the query's benchmark saves 12.2% over one median per candidate; our single probe, not told the benchmark, saves 31.8%, or 22.4% [16.5, 27.6] beyond benchmark identity, with intervals excluding zero in all ten training sets. MixLLM, whose encoder is fine-tuned to separate the benchmarks during training but reads only the query at test time, saves 8.8% beyond benchmark identity; ours saves 14.9% [9.9, 20.0] beyond MixLLM. An ablation asks whether the probe reads more than benchmark and difficulty: given the true benchmark, length from predicted success or from predicted difficulty (bins), fitted within each benchmark, saves about 24%, and the cost probe still saves 9.9% [5.2, 14.2] beyond it.

Prefill size. Savings do not grow with the prefill (Table 7): from 1.7B to 14B, every Qwen3 prefill saves 24.9–28.8% on LCB, 27.9–30.7% on Omni-MATH and 19.4–20.7% on MMLU-Pro relative to median pricing with its own success probes, while its forward pass costs about seven times more at 14B than at 0.6B. Counting each router's own pass, ours (4B) spends significantly less than the 14B router on LCB and

	LiveCodeBench			Omni-MATH			MMLU-Pro		
Prefill	vs. median	$/1k	Ours vs. it	vs. median	$/1k	Ours vs. it	vs. median	$/1k	Ours vs. it
Qwen3-0.6B	24.1	.005	3.5	27.9	.002	6.5	17.0	.002	4.8
Qwen3-1.7B	27.6	.007	−1.6	29.0	.003	3.4	19.4	.004	2.5
Qwen3-4B	25.5	.015	2.7	28.1	.006	3.5	20.6	.007	3.2
Qwen3-8B	24.9	.022	3.6	30.7	.008	0.6	20.7	.011	**6.2**
Qwen3-14B	28.8	.036	**7.3**	27.9	.014	9.8	20.1	.018	**10.5**
Qwen3-4B-2507 (ours)	27.8	.015	–	31.5	.006	–	23.4	.007	–

Table 7: **Prefill size.** Qwen3 prefills from 0.6B to 14B (hybrid releases) and ours (4B Instruct-2507), each feeding both probes. vs. median: cost saved (%) relative to median pricing with the row's own success probes. $/1k: the prefill's forward pass per 1,000 queries on one H100 at $2 per hour. Ours vs. it: cost saved (%) by our router relative to the row's, each paying for its own prefill, at our spend in the middle of the accuracy band (one training set); bold: 95% interval excludes zero. Off-family prefills in Table 17.

MMLU-Pro (by 7.3% [2.2, 11.2] and 10.5% [2.3, 15.5]) and than the 8B router on MMLU-Pro (6.2% [1.1, 12.4]). A 0.6B prefill reads length worse (ours is ahead by 7.3–8.2 points before its cheaper pass is counted), and a 1.7B prefill does about as well as ours. Prefills from three other families save 16.9–21.4% on MMLU-Pro and 25.1–28.2% on Omni-MATH (Table 17).

9 Controls

A text-embedding model of the same size. Reading cost from Qwen3-Embedding-4B (Zhang et al., 2025), a text-embedding model of the prefill's size, with any of three probes and our success predictions, or building a fully text-based router on it, saves less than our prefill router: by 9.7–23.1% (embedding cost probes) and 6.8–18.5% (text router) on LCB, Omni-MATH, MMLU-Pro and AIME, with intervals excluding zero, and on average on the other five datasets, where it is never significantly ahead (Table 16).

Is fine-tuning needed? Fully fine-tuning DistilBERT 66M, jina-embeddings 137M or Qwen3-0.6B on the same length labels, as CARROT does with RoBERTa (Somerstep et al., 2025), saves less than our probe on Omni-MATH and MMLU-Pro (by 11.9–25.6%, one training set) and on LCB (by 6.5% [−1.2, 12.2] for Qwen3-0.6B); the same Qwen3-0.6B, frozen and read by a linear probe, beats its fine-tuned copy, consistent with full fine-tuning distorting good pretrained features (Kumar et al., 2022) (Appendix J).

Is the prefill router's success pipeline needed? At our training sizes (a few hundred problems per benchmark, far fewer than the prefill router was trained on), our logistic probes on fixed layers tie a reimplementation of its success pipeline (Varshney et al., 2026) (per-target layer, pooling and PCA selection, then an MLP ensemble) on four of five datasets and trail it only on SuperGPQA, where neither predicts success well (one training set; Appendix J). With the same-size text embedding predicting success worse on every dataset, no success predictor we tried is better than ours on more than one dataset.

10 Deployment

Routing versus single models. A random mix of candidates with fixed probabilities reaches any point on the convex hull of the candidates' cost–accuracy points without a router; it is the baseline a router has to beat (Hu et al., 2024). At matched accuracy our router spends 22.9–39.2% less than the mix on LCB, Omni-MATH and MMLU-Pro and 14.9–46.6% less on AIME, SuperGPQA and BIG-Bench Extra Hard, and ties it on the other three datasets. No other estimator beats the mix on those three either, and on BigCodeBench neither does perfect cost knowledge. With median pricing the same success probes do not beat the mix on MMLU-Pro or AIME: there the cost estimate decides whether routing pays at all.

At the top of the frontier the cost estimate stops mattering: the most accurate policy sends each query to the candidate with the highest predicted success, so it is the same for every cost estimator. deepseek-v4-flash is

the most accurate candidate or within 1.6 points of it on every dataset, and the router's most accurate policy ends 0.2–2.7 points below the most accurate candidate on seven datasets; large-scale evaluations likewise find routers close to assigning each dataset its best single model (Li et al., 2026a). No success predictor we tried is better than ours on more than one dataset (Section 9), and larger ones are unlikely to pay for themselves here: a verifier needs the answer first, and even a free, perfect one does not let a cascade match one-shot routing on LiveCodeBench or Omni-MATH, while our 4B pass already costs 2–30% of the saving and savings grow little from a 0.6B to a 4B prefill (Appendix K). Below the top, giving up five points of accuracy saves 14.9–40.3% relative to deepseek-v4-flash alone on six datasets, 1.5–1.8× what median pricing saves. Under a hard cap on each call's cost, the router beats deepseek-v4-flash alone at the tightest cap we tested (by 8.2 and 7.0 points of accuracy on LCB and MMLU-Pro) and ties or trails it by up to 3 points at looser caps (Appendix K).

Deployable policies and the router's own cost. A deployed router must fix V in advance. Choosing, for each of ten accuracy targets, the cheapest V that reaches it on calibration, our deployed policies save 19.6–37.9% relative to median pricing's on six datasets, cost 1.0–3.3% more than our own test frontier and miss their target by 2.2–5.3 points (Table 18). The features come from one 4B forward pass over the prompt (19–28 ms on one H100), which a prefill router that predicts success already pays; the cost probe adds nothing to it (Appendix K).

11 Discussion and Limitations

When does per-query cost pay? For reasoning candidates perfect cost knowledge is worth 29–61% on eight of nine datasets, and much less for non-reasoning candidates on the same problems; how much it is worth on a new dataset is not predictable from how widely lengths and prices vary, but it is cheap to measure. The contrast agrees with CARROT, which found that per-query cost adds little over each model's average cost on RouterBench, whose eleven models include no reasoning model (Hu et al., 2024), and on SPROUT, where one of fourteen is (o3-mini) (Somerstep et al., 2025). The saving comes from deciding, query by query, whether a strong candidate whose cost varies widely is worth calling instead of a cheap one. For reasoning candidates the variation depends mostly on the problem, so what limits capture is the estimator: where length tracks difficulty, even length from predicted difficulty captures most of it; where it does not, only the cost probe does among the estimators we tried, and capture on MMLU-Pro is low (.40), leaving room for better cost predictors. One-shot software-engineering patches behave like the coding datasets, while for agents, whose prices span two orders of magnitude, choosing the model dominates (Appendix L).

Generalisation. In-domain and with full training data, a router could take length from predicted difficulty and lose little on most datasets. The case for a dedicated cost probe is that it holds up where a deployed router actually operates: with few labels, with new models, on new benchmarks and in mixed traffic, where the text and item-response estimators fall further behind it and length from predicted difficulty holds up only on benchmarks like those it was trained on (Section 8). A benchmark unlike its sources still needs some labels: on Omni-MATH and BIG-Bench Extra Hard the transferred router saves little.

Limitations. Each candidate needs output-length labels, though a few dozen suffice for most datasets. Exceeding the most accurate single candidate would take better success predictions than ours; the cost probe does not change the top of the frontier. At these model prices a larger success predictor would cost a large share of the saving it is meant to improve, but the case for one grows with the price of the routed models. Telling apart two strong candidates of similar price is the open part of the problem: billing gpt-oss-120b at high effort at deepseek-v4-flash's token rates, perfect per-query cost knowledge would save 11–36% within that pair, but our router saves at most 7.0% [2.4, 14.0] (Omni-MATH) relative to calling deepseek-v4-flash alone at its accuracy. The saving also needs a cheap candidate that solves a good share of problems. The main experiments use five candidates from two model families, with three further families on three datasets, and transfer is evaluated across benchmarks that share the routed models. Intervals are pointwise and assume independent problems, and a few analyses use one training set. The Omni-MATH and MMLU-Pro test sets have one draw per problem and candidate. Of the five main candidates only deepseek-v4-flash is

pinned to a provider; billed rates come from one collection, and prices change within days. The ZeroRouter, MixLLM and CARROT baselines are reimplementations.

12 Conclusion

A frozen prefill that already predicts success also predicts cost. Per-query cost is worth predicting for reasoning models, whose output lengths vary widely within a model, and much less for non-reasoning ones on the same problems. On held-out test problems at billed prices, the cost probe reduces spending at matched accuracy on eight of nine datasets, and no alternative estimator we tested is significantly cheaper on any of them; length from predicted difficulty matches it only where output length tracks difficulty, and only in domain. With few labels and for new models the cost probe keeps most of its saving, and on four of seven new benchmarks it does so without any target labels; in mixed traffic it saves a fifth of the cost beyond pricing by each query's benchmark.

Reproducibility

Every number in the paper is computed from stored per-call records (prompt, response, grade, token counts, billed cost and serving provider) and stored prefill activations, with every split fixed by a seed; no analysis issues a new model call. Appendix B gives the evaluation protocol, Appendix B.5 the reimplemented estimators, and Appendix A the datasets, splits and billed rates. Collection used OpenRouter with deepseek-v4-flash pinned to one provider; prices and provider behaviour change over time, so rerunning collection will not reproduce billed costs exactly, while every analysis reproduces from the released records.

TODO: Confirm what is released (code, per-call records, prefill features) and add the anonymised link.

Acknowledgements

This research was funded in part by Mitacs through the Mitacs Accelerate program and from the direct support of ServiceNow. Compute was provided by ServiceNow Research.

References

Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, et al. AutoMix: Automatically Mixing Language Models. In *Advances in Neural Information Processing Systems*, 2024.

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2016.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, et al. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model, 2025. URL `https://arxiv.org/abs/2502.02737`. arXiv:2502.02737.

Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *arXiv preprint arXiv:2505.20411*, 2025.

Dylan Bouchard. Is Escalation Worth It? On the Depth of LLM Cascades, 2026. URL `https://arxiv.org/abs/2605.06350`. arXiv:2605.06350.

Qi Cao, Shuhao Zhang, Ruizhe Zhou, et al. Models Under SCOPE: Scalable and Controllable Routing via Pre-hoc Reasoning, 2026. URL `https://arxiv.org/abs/2601.22323`. arXiv:2601.22323.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance, 2023. arXiv:2305.05176.

DeepSeek-AI. DeepSeek-V4-Flash, 2026. Model card, `https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash`.

Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A Unified Approach to Routing and Cascading for LLMs, 2024. arXiv:2410.10347.

di-zhang-fdu. AIME 1983–2024, 2024. Dataset, https://huggingface.co/datasets/di-zhang-fdu/AIME_1983_2024.

Naihua Duan. Smearing estimate: A nonparametric retransformation method. *Journal of the American Statistical Association*, 78(383):605–610, 1983.

Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A Graph-based Router for LLM Selections, 2024. arXiv:2410.03834.

Bofei Gao, Feifan Song, Zhe Yang, et al. Omni-MATH: A Universal Olympiad Level Mathematic Benchmark For Large Language Models, 2024. URL https://arxiv.org/abs/2410.07985. arXiv:2410.07985.

Irena Gao, Percy Liang, and Carlos Guestrin. Model equality testing: Which model is this API serving? In *International Conference on Learning Representations*, 2025.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 Herd of Models, 2024. URL https://arxiv.org/abs/2407.21783. arXiv:2407.21783.

Michael Günther, Jackmin Ong, Isabelle Mohr, et al. Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents, 2023. URL https://arxiv.org/abs/2310.19923. arXiv:2310.19923.

Liang He, Jingbo Wen, Yixiong Chen, Yue Yang, Qizhen Lan, Kangning Cui, and Xilu Wang. You cannot pick a provider from the price list: Market-aware routing for open-weight LLM inference, 2026.

Dan Hendrycks, Steven Basart, Saurav Kadavath, et al. Measuring Coding Challenge Competence With APPS, 2021. URL https://arxiv.org/abs/2105.09938. arXiv:2105.09938.

Qitian Jason Hu, Jacob Bieker, Xiuyu Li, et al. RouterBench: A Benchmark for Multi-LLM Routing System, 2024. URL https://arxiv.org/abs/2403.12031. arXiv:2403.12031.

Naman Jain, King Han, Alex Gu, et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code, 2024. URL https://arxiv.org/abs/2403.07974. arXiv:2403.07974.

Mehran Kazemi, Bahare Fatemi, Hritik Bansal, et al. BIG-Bench extra hard. *arXiv preprint arXiv:2502.19187*, 2025.

Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations*, 2022.

Janghoon Lee. Most of the LLM Routing Gap Is Task Type, 2026. arXiv:2608.23023.

Hao Li, Yiqun Zhang, Zhaoyan Guo, et al. LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing, 2026a. arXiv:2601.07206.

Haorui Li et al. When is the same model not the same service? a measurement study of hosted open-weight LLM APIs, 2026b.

Yujia Li, David Choi, Junyoung Chung, et al. Competition-Level Code Generation with AlphaCode, 2022. URL https://arxiv.org/abs/2203.07814. arXiv:2203.07814.

Guanjie Lin, Yinxin Wan, Shichao Pei, Ting Xu, Kuai Xu, and Guoliang Xue. Behavioral consistency and transparency analysis on large language model API gateways, 2026.

William Lugoloobi, Thomas Foster, William Bankes, and Chris Russell. LLMs Encode Their Failures: Predicting Success from Pre-Generation Activations, 2026. arXiv:2602.09924.

M-A-P Team. SuperGPQA: Scaling LLM evaluation across 285 graduate disciplines. *arXiv preprint arXiv:2502.14739*, 2025.

Glen Messenger. ALPS: Activation-Based Length Prediction for Intelligent LLM Inference Scheduling, 2026. URL https://doi.org/10.5281/zenodo.19078431. Zenodo. doi:10.5281/zenodo.19078431.

Microsoft. Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs, 2025. URL https://arxiv.org/abs/2503.01743. arXiv:2503.01743.

Nebius. SWE-agent trajectories, 2024. Dataset, https://huggingface.co/datasets/nebius/SWE-agent-trajectories.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, et al. RouteLLM: Learning to Route LLMs with Preference Data, 2024. arXiv:2406.18665.

OpenAI. gpt-oss-120b & gpt-oss-20b Model Card, 2025. URL https://arxiv.org/abs/2508.10925. arXiv:2508.10925.

Zhihong Pan, Kai Zhang, Yuze Zhao, and Yupeng Han. Route to Reason: Adaptive Routing for LLM and Reasoning Strategy Selection, 2025. arXiv:2505.19435.

Shivam Patel, Akaash R. Parthasarathy, Ankur Mallick, and Gauri Joshi. Beyond Accuracy and Cost: Latency-Aware LLM Query Routing for Dynamic Workloads, 2026. URL https://arxiv.org/abs/2607.18253v1. arXiv:2607.18253v1.

John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pp. 61–74. MIT Press, 1999.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, 2019. URL https://arxiv.org/abs/1910.01108. arXiv:1910.01108.

Seamus Somerstep, Felipe Maia Polo, Allysson Flavio Melo de Oliveira, et al. CARROT: A Cost Aware Rate Optimal Router, 2025. URL https://arxiv.org/abs/2502.03261. arXiv:2502.03261.

Seongho Son, Sangwoong Yoon, Jiahua Tang, Shuhan Wang, et al. SWE-Router: Routing in Multi-turn Agentic Software Engineering Tasks, 2026. arXiv:2607.00053.

Wei Song, Zhenya Huang, Cheng Cheng, Weibo Gao, et al. IRT-Router: Effective and Interpretable Multi-LLM Routing via Item Response Theory, 2025. arXiv:2506.01048.

Ihor Stepanov, Aleksandr Smechov, Mykhailo Shtopko, et al. SCX Router: Streaming Zero-Shot Model Selection with a Decoder-KV Classifier and a Real-World Task Ontology, 2026. URL https://arxiv.org/abs/2609.02292. arXiv:2609.02292.

Tanay Varshney, Annie Surla, Michelle Xu, et al. LLM Router: Rethinking Routing with Prefill Activations, 2026. URL https://arxiv.org/abs/2603.20895. arXiv:2603.20895.

Wenhui Wang, Furu Wei, Li Dong, et al. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers, 2020. URL https://arxiv.org/abs/2002.10957. arXiv:2002.10957.

Xinyuan Wang, Yanchi Liu, Wei Cheng, et al. MixLLM: Dynamic Routing in Mixed Large Language Models, 2025. URL https://arxiv.org/abs/2502.18482. arXiv:2502.18482.

Yubo Wang, Xueguang Ma, Ge Zhang, et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.

Weihuang Wen, Yingying Liu, Yichuan Liu, et al. OUTLETS: Output-Length Prediction from Speculative Decoding Backbones, 2026. URL https://arxiv.org/abs/2609.01068. arXiv:2609.01068.

Huanyi Xie, Yubin Chen, Liangyu Wang, et al. Predicting LLM Output Length via Entropy-Guided Representations, 2026. URL https://arxiv.org/abs/2602.11812. arXiv:2602.11812.

Jiaqi Xue, Qian Lou, Jiarong Xing, et al. R2-Router: A New Paradigm for LLM Routing with Reasoning, 2026. URL https://arxiv.org/abs/2602.02823. arXiv:2602.02823.

Cheng Yan, Wuyang Zhang, Zhiyuan Ning, et al. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space, 2026. URL https://arxiv.org/abs/2601.06220. arXiv:2601.06220.

An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 Technical Report, 2025a. URL https://arxiv.org/abs/2505.09388. arXiv:2505.09388.

John Yang, Kilian Lieret, Carlos E. Jimenez, et al. SWE-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025b.

Yanzhao Zhang, Mingxin Li, Dingkun Long, et al. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models, 2025. URL https://arxiv.org/abs/2506.05176. arXiv:2506.05176.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *International Conference on Learning Representations*, 2025.

A Pools

Dataset	Train / calib. / test problems	Draws per candidate (train)	Draws per candidate (test)	Test acc. (%)	Median output per candidate	90th/10th pct. ratio
LiveCodeBench (Jain et al., 2024)	441 / 110 / 341	16/12/8/6/3	16/12/8/6/3	52–90	0.7–7.0k	11–121
Omni-MATH (Gao et al., 2024)	275 / 75 / 1,000	4/3/3/2/2	1/1/1/1/1	42–71	1.4–10.5k	10–126
MMLU-Pro (Wang et al., 2024)	550 / 150 / 6,500	4/3/3/2/2	1/1/1/1/1	62–86	0.2–1.6k	7–29
AIME 1983–2024 (di-zhang-fdu, 2024)	513 / 139 / 281	4/3/3/2/2	4/3/3/2/2	57–98	1.8–7.7k	4–16
APPS (Hendrycks et al., 2021)	489 / 123 / 377	1/1/1/1/1	1/1/1/1/1	58–86	0.6–10.3k	10–91
BigCodeBench (Zhuo et al., 2025)	544 / 136 / 421	16/12/8/6/3	16/12/8/6/3	45–53	0.4–2.1k	2–4
CodeContests (Li et al., 2022)	346 / 86 / 266	4/3/3/2/2	4/3/3/2/2	49–86	0.8–13.1k	5–37
SuperGPQA (M-A-P Team, 2025)	550 / 150 / 300	4/3/3/2/2	4/3/3/2/2	31–68	0.2–3.3k	7–25
BIG-Bench Extra Hard (Kazemi et al., 2025)	550 / 150 / 300	4/3/3/2/2	4/3/3/2/2	23–65	0.4–16.4k	14–26

Table 8: **Pools.** Split sizes (problems with a valid answer from every candidate), median draws per candidate (in Table 2's order), and ranges over candidates of test accuracy, median output length (tokens) and the 90th/10th-percentile length ratio. Omni-MATH and MMLU-Pro train on Omni-MATH-500 and a 1,000-question subject-stratified sample and test on 1,000 and 6,500 further problems; the other datasets use a random held-out split. Billed rates (USD per million input / output tokens): gpt-oss-20b 0.029 / 0.134, deepseek-v4-flash (StreamLake) 0.042 / 0.084, gpt-oss-120b 0.020 / 0.257.

B Implementation details

B.1 Data, candidates and labels

All calls go through OpenRouter. deepseek-v4-flash is pinned to StreamLake by restricting OpenRouter's provider routing to that provider with fallbacks disabled; the gpt-oss candidates are served by whichever provider OpenRouter selects. LiveCodeBench has 892 problems (contests from September 2023 to April 2025), split at random into 441 training, 110 calibration and 341 test problems. Omni-MATH and MMLU-Pro have 275/75 and 550/150 training/calibration problems, from the 500-problem Omni-MATH set and a 1,000-question subject-stratified sample of MMLU-Pro, and 1,000 and 6,500 disjoint test problems. The six further datasets are split at random (Table 8). Training problems have 2–16 draws per candidate; Omni-MATH and MMLU-Pro test problems have one. Every draw is scored by the benchmark's own grader (code execution against the tests for the coding datasets; answer matching for the others), and evaluated problems need a valid answer from every candidate.

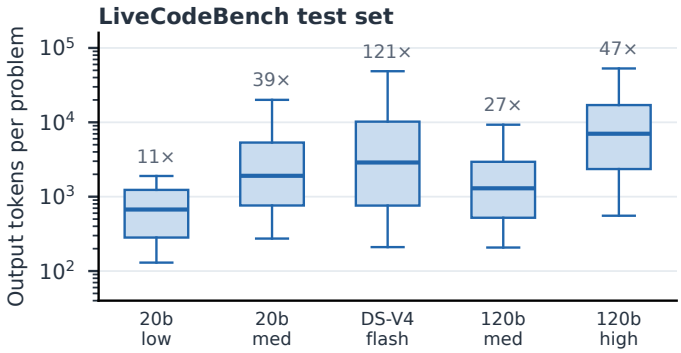


Figure 3: **Output length varies by orders of magnitude within one model.** Per-problem output tokens by candidate on the LiveCodeBench test set (log scale). Boxes: interquartile range; whiskers: 5th–95th percentile; line: median; above each box: 90th/10th-percentile ratio.

B.2 Prices

Effective rates are least-squares fits of each call's billed cost on its input and output tokens, per model (Table 8). For the gpt-oss models they average over the providers OpenRouter chose, which billed gpt-oss-120b from 0.17 to 0.60 USD per million output tokens. A problem's realized cost on a candidate is the mean billed cost of its valid draws on the Omni-MATH and MMLU-Pro test sets, and the mean of its valid draws' input and output tokens at the effective rates elsewhere. Two checks over ten training sets: fitting the rates on training calls only (the same for deepseek-v4-flash and gpt-oss-120b, a 15% higher output price for gpt-oss-20b), and costing the Omni-MATH and MMLU-Pro test sets as tokens at the effective rates. Each moves our saving relative to median pricing by at most 1.0 point, and no comparison in Table 9 flips sign or loses significance. SuperGPQA and BIG-Bench Extra Hard were collected later, when deepseek-v4-flash was billed at about twice its earlier rate; we price them like every other dataset (at their own billed rates our saving there would be 4.9 and 1.1 points higher, on one training set).

B.3 Probes

The features are the mean- and last-token activations of eight layers of Qwen3-4B (Instruct-2507; Thinking-2507 on Omni-MATH), concatenated and shifted and scaled to zero mean and unit variance on the training problems. Success probes are per-candidate L2 logistic regressions, with the regularization chosen on calibration AUC from nine values and Platt scaling fitted on calibration. Cost probes are per-candidate ridge regressions of log mean output length, with the regularization chosen by cross-validation within the training problems; the exponentiated prediction is corrected by Duan's smearing from the training residuals and its level matched to the candidate's training mean.

B.4 Evaluation

Frontiers and matched accuracy. Each value V on a geometric grid of 300 values gives a deterministic policy (Eq. (1)) and a test point (mean realized cost, mean accuracy). An arm's frontier is the upper concave hull of its points, and its cost at a target accuracy is linearly interpolated on the hull. Two arms are compared at 12 accuracies evenly spaced over the interior 90% of the accuracy range both frontiers reach; the reported saving is one minus the geometric mean of their cost ratios. Which problems are used for training matters at these sizes, so every result is a mean over ten training sets. A training set reshuffles the training and calibration problems (with a fixed seed per training set) into splits of the same sizes and keeps the test problems; the success probes and every estimator are refitted and every configuration is re-chosen on its calibration problems. Within a training set, intervals resample test problems with replacement (300 paired resamples, the same for both arms and for every training set, all candidates of a problem kept together) with that set's probes fixed; reported intervals are percentiles of the 3,000 resamples pooled over the ten

training sets, so they reflect both the choice of training problems and the test problems, and reported values are means of the ten point estimates.

Analyses on one training set. Every table and every number in the main text is a mean over the ten training sets unless marked "one training set". Those marked use one fixed split of each dataset and compare arms within it: held-out draws, rescaled prices, the within-test variance decomposition and length drivers of Section 7, fine-tuned encoders on Omni-MATH and MMLU-Pro, parts of the transfer analysis, the prefill router's success pipeline, the perfect-judge cascade and the software-engineering agent analyses.

B.5 Reimplemented estimators

Every estimator predicts each candidate's log mean output length from information available before generation, is fitted on the training problems only, and is post-processed like our probe (residual smearing, level matched to the candidate's training mean); configurations are chosen on the calibration split. *Length from predicted success*, our ablation, fits each candidate's log length by ridge regression on the success logits of all five candidates and their squares, so it reads the prefill only through the success probes. *Prompt-feature GBM* follows latency-aware routing (Patel et al., 2026): histogram gradient boosting (300 iterations, learning rate 0.05, minimum leaf 10) on 44 prompt statistics (length, line count, count and magnitude of numbers, math delimiters, examples, constraint markers, code fences, inequality signs and 34 keyword counts), with scikit-learn's histogram boosting in place of LightGBM. *MixLLM-style* (Wang et al., 2025): a jina-embeddings-v2 137M embedding (Günther et al., 2023) of the prompt (the code variant on LiveCodeBench), read by the average of an MLP, a random forest and a k-nearest-neighbour regressor per candidate; MixLLM fine-tunes a BERT encoder. *MixLLM* (Wang et al., 2025) as published, in mixed traffic only: bert-base-uncased, mean-pooled, fine-tuned with the paper's domain loss (each embedding pulled toward its domain's centre, centres pushed apart), with the nine benchmarks as domains in place of InsTag tags grouped by hand; the paper's tags need a 13B tagger and give sub-topics our benchmarks do not label, and within one benchmark the loss has nothing to separate, so the per-dataset tables use the MixLLM-style version. The paper gives no normalization or optimizer settings, so embeddings and learnable centres are normalized with temperature 0.1 (AdamW, learning rate $2 \cdot 10^{-5}$, batch 32, up to three epochs, the epoch chosen on calibration log-length error). Each candidate's length regressor is the MLP, random forest or k-nearest-neighbour regressor with the lowest calibration error (the random forest in 48 of 50 cases). Fine-tuning helps: the same pipeline on pre-trained BERT saves 6.4% beyond benchmark identity, against 8.8%, and with MixLLM's own random-forest success model in place of ours it saves 10.7% (ours 13.1% [6.7, 18.6] beyond it). *CARROT-style kNN* (Somerstep et al., 2025): the authors' local variant, MiniLM-L12 (Wang et al., 2020) sentence embeddings with uniform cosine kNN, k chosen by five-fold cross-validation on training problems. *Route-To-Reason* (Pan et al., 2025), as published: a frozen all-mpnet-base-v2 embedding of the query, concatenated with a fixed embedding of a short text description and a learned 64-dimensional embedding of the model and of its effort (the paper's reasoning strategy), read by one MLP shared by all candidates (hidden size 768) trained with squared error on output length (Adam, learning rate 10^{-3}, batch 32, up to 100 epochs, early stopping on calibration loss); the same network on log length saves within 5 points of it on every dataset. *One length per query (SCX-style)* (Stepanov et al., 2026): SCX Router reads one expected output length per task rather than per model; we read one per query from our prefill (ridge onto the candidate-centred mean log length) and add each candidate's training mean. *ZeroRouter* (Yan et al., 2026): stage 1 fits a D-dimensional two-parameter item-response model to the five candidates' training outcomes (MAP with Gaussian priors; the paper uses stochastic variational inference over about 200 leaderboard models). Stage 2 places a new query in this latent space by regressing its item parameters $(\log \alpha, b)$ on the query. *As published*, stage 2 fine-tunes DistilBERT (Sanh et al., 2019) with 11 linguistic features of the prompt (40 epochs, batch 32, learning rate $3 \cdot 10^{-5}$, squared error on the standardized parameters, the epoch chosen on a held-out tenth of the training problems; the paper does not list its features, so ours are length, word and sentence statistics, two readability scores, the shares of digits and math symbols, and bracket depth). *With our 4B prefill*, stage 2 is a ridge regression on our prefill features after PCA to 256 dimensions. Both use the paper's success model $\sigma(\alpha^\top(\theta_m - b))$ and its pricing, the candidate's mean training length within K quantile bins of $s = \alpha^\top b$, with $D \in \{1, 5\}$ and $K \in \{5, 10, 20\}$ chosen on calibration. Both versions were also fitted together with our cost probe in one run on the ten training sets, where the 4B-prefill version's gaps match Table 1

to within 0.4 points; that run gives the as-published column. The 4B prefill is the better reader on every dataset (success AUC .55–.81 against .52–.73), but it changes ZeroRouter's gap to ours by at most 5.4 points: given our success probes, its bin pricing still trails our cost probe by 9.2–34.8 points on LCB, Omni-MATH, MMLU-Pro, AIME, SuperGPQA and BIG-Bench Extra Hard with our prefill (14.2–35.5 with DistilBERT), so its shortfall is in the pricing. Elsewhere in the paper ZeroRouter means the 4B-prefill version, the stronger of the two. With five candidates a higher-dimensional latent does not improve its success predictions, and on MMLU-Pro adding up to 53 Open LLM Leaderboard models to stage 1 did not either. The fine-tuned encoders of Section 9 are mean-pooled with a linear head on per-candidate standardised log length, trained with masked squared error (AdamW, 6% warm-up then linear decay, maximum 512 tokens for DistilBERT and 2,048 for the others); learning rate (two values per encoder) and epoch (up to 10, or 20 when the best epoch was the last) are chosen on calibration error, and smearing and level match use the calibration problems.

B.6 Experiment settings

Non-reasoning candidates. deepseek-v4-flash with thinking disabled uses the same provider and sampling as with thinking; the four other non-reasoning models are each pinned to one provider. Every non-reasoning candidate has one draw per problem, with output capped at 16k tokens. Qwen3-235B-A22B-Instruct has training answers for 241 of 275 Omni-MATH and 485 of 550 MMLU-Pro problems (rate limits).

Few labels. Each cost estimator is refitted on n random training problems, two subsets in each of the ten training sets, with the same success probes for every arm. ZeroRouter pricing fits its item-response latent on the n problems, reads it from the prefill and uses its bin lookup; the kNN neighbour count and ZeroRouter's dimension and bin count are chosen on calibration. Ours exceeds every external estimator with paired intervals excluding zero on LCB and Omni-MATH from n=20 (except Omni-MATH at n=50), on MMLU-Pro from n=100 and on SuperGPQA from n=200. On BIG-Bench Extra Hard the text estimators stay within noise of ours at every n; below 100 problems ours leads on average (by 16.5 points over MixLLM-style embeddings at 50), with intervals that include zero. Labelling one sampled output per training problem instead of all of them gives up 1–3 points (one training set).

Onboarding. The held-out model is gpt-oss-20b (both efforts), gpt-oss-120b (both efforts) or deepseek-v4-flash, so that no sibling effort remains in the pool, with four random choices of the k anchors in each training set, the same in every arm. ZeroRouter's onboarding fits the new model's ability on the k anchors in an item-response latent fitted on the remaining candidates and fills its per-bin length table from them (Yan et al., 2026); the CARROT-style arm averages the anchors' outcomes and lengths over nearest neighbours. ZeroRouter's dimension and bin count and the kNN neighbour count are chosen on calibration.

Transfer. The domains are coding (LCB, APPS, BigCodeBench, CodeContests) and math and knowledge (Omni-MATH, MMLU-Pro, AIME, SuperGPQA, BIG-Bench Extra Hard). Every choice (regularization, Platt scaling, neighbour counts, latent dimension, bin count) is made on the source datasets' calibration splits, and each training set reshuffles every dataset's splits with the same seed. External estimators are transferred the same way, each with its own success model (MixLLM-style: a logistic probe on the text embedding; kNN; ZeroRouter: its latent and bins). Omni-MATH enters as its 500-problem set (test 150), whose prefill matches the other datasets'. BigCodeBench and CodeContests, where no in-domain estimator saves significantly, are not used as targets: there the transferred success probes are badly miscalibrated (BigCodeBench log loss 1.17 against .67 in-domain) and the apparent gains are not robust. Transferred success predictions are weaker than in-domain ones (LCB AUC .67 against .83), so the transferred router's saving comes from its cost probe.

Mixed traffic. Training and calibration problems are the union of each dataset's splits; the test set has 150 problems from each dataset (1,350); Omni-MATH enters as its 500-problem set. Both probes are refitted on the mixture, and bootstrap resamples are drawn within each benchmark.

C Intervals on LiveCodeBench, Omni-MATH and MMLU-Pro

Cost saved by ours vs.	LCB (n=341)	Omni-MATH (n=1,000)	MMLU-Pro (n=6,500)
Median pricing	27.8 [13.9, 35.6]	31.5 [23.9, 37.7]	23.4 [17.1, 29.0]
Mean pricing	27.8 [19.5, 34.6]	34.3 [26.3, 40.1]	25.8 [20.4, 30.6]
Prompt-feature GBM	20.7 [10.1, 29.0]	25.8 [15.9, 35.0]	20.0 [13.9, 25.9]
MixLLM-style embeddings	16.2 [7.9, 22.6]	24.7 [15.9, 31.7]	15.5 [10.6, 20.5]
ZeroRouter (as published, DistilBERT)	23.5 [4.7, 38.2]	19.6 [3.0, 31.3]	19.5 [6.3, 25.8]
ZeroRouter (4B prefill)	19.3 [7.7, 28.2]	14.2 [−4.7, 39.3]	18.4 [5.9, 24.2]
Headroom: oracle vs. median	40.1 [29.0, 46.9]	51.9 [44.3, 56.8]	58.0 [53.9, 60.8]

Table 9: **Cost saved at matched accuracy, billed costs.** Cost saved (%) by our cost probe relative to each estimator, with 95% intervals. ZeroRouter uses its own success model; ZeroRouter (4B prefill), the stronger version, reads its item parameters from our prefill. All intervals exclude zero except against ZeroRouter (4B prefill) on Omni-MATH (ours ahead in 7 of 10 training sets). Last row: the headroom, of which the cost probe captures 69%, 61% and 40%. Estimators in Appendix B.5.

D Non-reasoning candidates

	Headroom		Ours vs. median		Capture	
Test set	reasoning	non-reas.	reasoning	non-reas.	reasoning	non-reas.
LiveCodeBench (341)	40.1	21.4	27.8	12.1 (5.4)	.69	.53
Omni-MATH (1,000)	53.1	40.5	31.5	6.8 (1.6)	.59	.17
MMLU-Pro (2,000)	59.8	26.0	25.8	0.8 (2.2)	.43	.03

Table 10: **Reasoning versus non-reasoning candidates on the same problems.** Headroom, our saving relative to median pricing (s.d. over training sets) and capture, for five reasoning and five non-reasoning candidates, on the test problems every candidate of both pools answers validly (MMLU-Pro: a 2,000-problem subset). The oracle prices each problem at its billed cost, so Omni-MATH's reasoning headroom differs slightly from Table 3. The non-reasoning saving excludes zero in every training set on LCB and Omni-MATH and in none on MMLU-Pro.

Adding the non-reasoning candidates to the reasoning pool lowers our saving (14.8% on Omni-MATH, 7.6% on MMLU-Pro, 19.2% on LCB). Non-reasoning lengths do vary (s.d. of log length .53–1.73 per candidate), but the variation has two sources that the prompt predicts poorly. Llama-3.1-8B, at its default sampling settings, falls into repetition until the 16k cap on 9–12% of math problems (four in five capped answers loop), so its mean output is 4–6× its median and its log-length R^2 is below zero on every set. This is the model rather than the provider: redrawn on 300 such problems, it loops again on 29% with the same provider and 19% with another, against about 5% on control problems for both; a repetition penalty of 1.1 removes most loops (6%), apparently at some cost in accuracy. Qwen3-30B-A3B-Instruct, asked for code only on LCB, reasons inside code comments on hard problems until the cap (9% of problems), so its lengths span 142× between its 10th- and 90th-percentile problems (R^2 .27): the largest non-reasoning headroom comes from a non-reasoning model that reasons anyway.

Every estimator on non-reasoning candidates. Non-reasoning candidates combine both failures (Table 10): less headroom, and on Omni-MATH and MMLU-Pro a capture of .17 and .03. No other estimator does better. Fitting every estimator of Tables 1 and 16 on the non-reasoning candidates (ten training sets), none of mean length, prompt features, MixLLM-style or nearest-neighbour embeddings, ZeroRouter's bins and the three Qwen3-Embedding-4B probes saves significantly relative to median pricing on either dataset (at most 1.9%); on Omni-MATH ours is ahead of each by 4.8–12.2 points with length from predicted success tied, and on MMLU-Pro every estimator, ours included, is within noise of median pricing. Most of their length is nonetheless readable: on Omni-MATH our probe explains .55–.75 of four candidates' log length,



Figure 4: **Candidate shares at matched accuracy** (one training set). Share of test queries (%) sent to each candidate at the middle of the accuracy band that all three pricings reach; right: spend per 1,000 queries. Candidates are ordered by mean cost per query; shares below 10% are unlabelled.

about what the other non-reasoning candidates' lengths on the same problem explain (.48–.70). The exception is Llama-3.1-8B, whose loops on math problems make its mean output 4–6× its median: no estimator reads its length (best R^2 .10–.22 across the three datasets) and neither do other models' lengths (.14–.40). What can be read is worth little here, and the most erratic candidate cannot be read at all. On LCB the probes capture .53 of a smaller headroom, with length from predicted success and the Qwen3-Embedding-4B ridge probe close behind (11.3% and 10.4% against 12.1%).

E Further mechanism results

All results in this appendix are on one training set.

Where the saving goes. At the middle of each dataset's shared accuracy band, our router sends fewer queries than median pricing to gpt-oss-20b at low effort, the candidate with the lowest typical cost, and more to deepseek-v4-flash or gpt-oss-120b at medium effort (Fig. 4); on every dataset but MMLU-Pro, SuperGPQA and BigCodeBench, pricing at realized length shifts traffic the same way. The mix is not the whole story: on MMLU-Pro and SuperGPQA, realized-length pricing sends about the same shares as median pricing at a third of its spend, so its saving lies in which queries go to each candidate rather than how many. gpt-oss-120b at high effort receives at most 1% of queries under any of the three.

The headroom is an upper bound. Pricing each problem at its realized cost also knows the sampling noise of the draws it is scored on, which no predictor of expected cost can. Where test problems have several draws per candidate, we split each problem's draws at random into a half that prices and a half that is scored (20 splits). On one training set, pricing at the held-out half's mean length, a target an expected-cost

predictor could reach, saves 41.2% on LiveCodeBench against 45.9% for the realized-cost oracle on the same halves; the gap is larger where draws are few (AIME 20.1 against 30.1, Omni-MATH-500 23.6 against 35.9, CodeContests 23.4 against 38.9) and small on SuperGPQA (52.7 against 57.1). With one or two draws per half the held-out oracle is itself noisy, so the achievable headroom lies between the two; on LiveCodeBench, where it is tightest, the cost probe captures .81 of the held-out bound against .72 of the realized one. The capture figures in Table 3 are therefore conservative.

Success and cost are read from the same layers. Refitting both probes on each stored layer separately, success AUC and length R^2 peak in the same late layers at the last token (layers 22–36 of 36), and one such layer matches the full concatenation (LCB AUC .86–.87, R^2 .73–.75, saving 33–34%; Omni-MATH .83–.84, .71–.72, 35–37%; MMLU-Pro .72, .48–.50, 27–29%). The two probes are close to one signal where length tracks difficulty: the mean success logit correlates with predicted log length at $-.90$ to $-.97$ per layer on LCB and $-.71$ to $-.84$ on Omni-MATH, but only $-.36$ to $-.58$ on MMLU-Pro.

What changes in the routing. At matched accuracy in the middle of the band, 31–66% of problems change candidate relative to median pricing. The dominant changes are between gpt-oss-20b-low and deepseek-v4-flash: problems predicted to be short move to the more accurate deepseek-v4-flash (19–38% of problems), and problems predicted to be long move to the cheap candidate (9–18%).

The success probes carry more than difficulty. Where length from predicted success ties the cost probe (LCB, Omni-MATH), it does not price by difficulty alone. Oracle difficulty, a problem's measured solve rate over the five candidates (with its square), known only after running and grading all of them, is a weak price even in-domain: it saves 21.5% on LCB against 34.4% for length from predicted success, because the logits carry length cues beyond the solve rate: among LCB problems that every other candidate always solves, the predicted logit still ranks length (Spearman $-.65$ to $-.85$).

Oracle difficulty and the prefill. Here oracle difficulty is a problem's solve rate across the other four candidates, so it never shares draws with the length it explains. Per candidate, we fit log length on the calibration problems from the prefill's prediction, from oracle difficulty (with its square), or from both, and score the test problems (one training set). Oracle difficulty alone explains a median R^2 of .57 on AIME, .54 on LCB, .45 on CodeContests, .38 on APPS and .33 on Omni-MATH, and .17 or less on BIG-Bench Extra Hard, MMLU-Pro, BigCodeBench and SuperGPQA. Adding it to the prefill raises R^2 by a median of .16 on AIME and .13 on CodeContests, and by .05 or less elsewhere.

Failed and solved draws. Within a problem, failed and solved draws are about equally long ($\times$0.83–1.16), with one exception: deepseek-v4-flash writes 2.5$\times$ (Omni-MATH) and 1.9$\times$ (AIME) longer when it fails on hard math. On MMLU-Pro, engineering and law questions are both long (6.4k and 4.1k output tokens on average) and hard.

The part of predicted length that difficulty does not explain. The part of the predicted length that oracle difficulty (the measured solve rates) does not explain is real. It correlates with the same part of the realized length at .56–.92 across six datasets (MMLU-Pro .81: engineering questions are predicted 0.95 log units longer than their difficulty implies, and are 0.95 longer). It also predicts the length of five held-out non-reasoning models on MMLU-Pro (partial correlation .40–.71) while saying almost nothing about whether the stronger of them succeed ($|r| \leq .05$ for deepseek-v4-flash without thinking and Qwen3-235B-Instruct).

F How much length depends on the problem, and what our probe reads

Table 11 gives, per candidate, three test R^2 values of log mean output length: our probe's, which is the highest of every pre-generation estimator we evaluated (length from predicted success, ZeroRouter's latent, prompt features, MixLLM-style and nearest-neighbour probes of the text encoder of Appendix B.5, ridge, MixLLM-style and nearest-neighbour probes of Qwen3-Embedding-4B, Route-To-Reason, and on LCB the fine-tuned Qwen3-0.6B) on every candidate of every dataset; that of a linear map from the mean realized log length of the other four candidates on the same problem, which is not a deployable estimator but shows

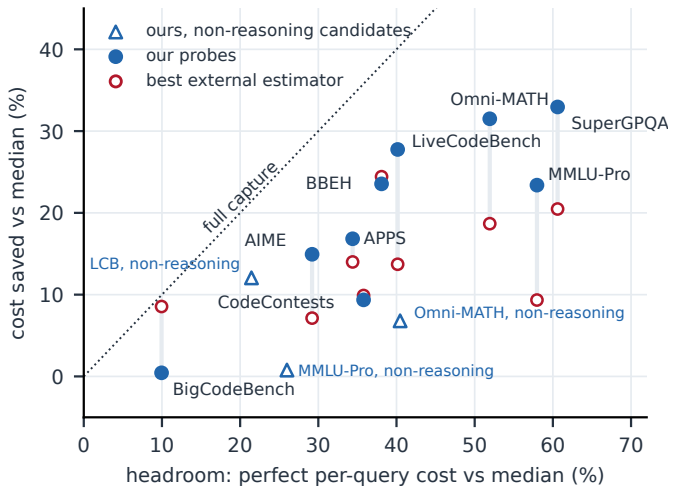


Figure 5: **Headroom and capture.** Per dataset, the headroom (x) against the saving of our cost probe (filled) and of the best external estimator on that dataset (hollow); triangles: our cost probe with five non-reasoning candidates (Table 10). The probe captures half or more of the headroom where it is large and length is predictable; BigCodeBench has little to capture, on CodeContests no estimator captures it, and with non-reasoning candidates no estimator captures more than about a sixth on Omni-MATH and MMLU-Pro.

how much of a candidate's length is shared with other models; and the between-problem share, the share of the variance of a candidate's per-problem log length over test problems that is between problems rather than between draws (from the candidates' repeated draws; APPS has one draw per problem). Means over ten training sets.

Table 12 decomposes log length with no predictor. Over problems, candidates and draws (problems with a valid draw on every candidate), after removing candidate means, a two-way random-effects decomposition (method of moments, harmonic-mean draw count) splits the variance into a shared problem level, a problem-by-candidate interaction and within-cell noise. Headroom by source compares oracles that know, for each test problem, its realized shared level only (with each candidate's training-mean offset), its candidate-specific part only (with the training-mean level), or both (the headroom of the main text); each is level-matched to the candidate's training mean like the probes.

We fixed four models of headroom in advance, fitted by least squares over the 138 candidate subsets of Appendix I (full pools, leave-one-out subsets and pairs), using only predictor-free statistics: (A) the price spread (s.d. over candidates of log mean cost per call), (B) the problem and interaction variances, (C) both, and (D) the log ratio of query-driven variance to squared price spread. In-sample R^2 is .32, .18, .37 and .11; leave-one-dataset-out R^2 is .25, −.23, .01 and −.01. Headroom depends on which candidates solve which problems, not only on how lengths and prices vary.

What our probe reads. On every candidate of every dataset our probe reads length better than length from predicted success, ZeroRouter's latent, prompt features, MixLLM-style and nearest-neighbour probes of a text encoder, three probes of Qwen3-Embedding-4B, Route-To-Reason and, on LCB, the fine-tuned Qwen3-0.6B (Table 11); one length per query, read from the same prefill, matches or slightly exceeds it on some candidates (by at most .04) but saves less (Table 1), because it fixes the ratio of lengths between candidates for every query. Our R^2 comes closest to the between-problem share on LCB, Omni-MATH and BIG-Bench Extra Hard (e.g. LCB .71–.75 against .98–.99) and falls furthest below it on the knowledge datasets, for deepseek-v4-flash above all: on MMLU-Pro and SuperGPQA we read .26 and .22 of its log length, other candidates' lengths explain .62 and .46, and the between-problem share is .92 and .96. Its length there depends on the question and is largely shared with other models, but the prefill misses it. Candidate differences are what predictors read worst: across candidate pairs, our R^2 for the log length ratio

is .06–.47 per dataset, against a between-problem share of .48–.85, and this differential R^2 tracks the share of a pair's headroom we capture (Spearman .49 over the 75 pairs with headroom above 5 points) better than per-candidate R^2 does (.18). On LCB (one training set), the prefill reads the level (R^2 .75) far better than the differences (.41), and pricing with the predicted level alone keeps 93% of the full probe's saving.

Dataset	20b low	20b med.	deepseek	120b med.	120b high
LiveCodeBench	.74 / .85 / .99	.71 / .96 / .99	.72 / .92 / .99	.73 / .95 / .99	.75 / .94 / .98
Omni-MATH	.68 / .60 / .84	.66 / .86 / .89	.71 / .79 / .95	.72 / .87 / .94	.73 / .88 / .95
MMLU-Pro	.61 / .42 / .71	.52 / .68 / .79	.26 / .62 / .92	.62 / .76 / .88	.45 / .67 / .89
AIME	.41 / .52 / .89	.44 / .85 / .93	.50 / .79 / .97	.49 / .86 / .92	.51 / .87 / .94
APPS	.56 / .69 / –	.54 / .80 / –	.63 / .86 / –	.63 / .88 / –	.68 / .88 / –
BigCodeBench	.69 / .66 / .97	.61 / .86 / .96	.47 / .71 / .95	.71 / .85 / .97	.45 / .67 / .88
CodeContests	.46 / .67 / .92	.35 / .83 / .93	.52 / .80 / .96	.42 / .88 / .94	.43 / .86 / .95
SuperGPQA	.75 / .50 / .94	.47 / .64 / .88	.22 / .46 / .96	.63 / .77 / .92	.34 / .54 / .90
BIG-Bench Extra Hard	.56 / −.01 / .81	.81 / .82 / .89	.83 / .72 / .96	.90 / .75 / .92	.84 / .69 / .94

Table 11: **How much of each candidate's length we read, and how much depends on the problem.** Test R^2 of log mean output length: our probe / a linear map from the other candidates' realized lengths on the same problem / the share of variance between problems rather than between draws. –: one draw per problem.

	Share of log-length variance (%)				Headroom by source (%)		
Dataset	Problem	Interaction	Noise	Corr.	Level only	Candidate-specific only	Both
LiveCodeBench	84	8	8	.94	25.5	24.8	40.1
Omni-MATH	80	12	8	.85	23.3	41.4	51.9
MMLU-Pro	66	17	17	.70	25.4	43.7	58.0
AIME	70	15	16	.79	15.6	23.8	29.2
APPS	81	19	–	.86	18.3	24.6	34.4
BigCodeBench	57	15	28	.80	−0.4	8.7	10.0
CodeContests	73	13	14	.83	18.8	28.9	35.8
SuperGPQA	55	27	18	.67	43.3	54.1	60.6
BIG-Bench Extra Hard	42	29	29	.49	20.9	31.5	38.1

Table 12: **Where the routing value lies, without a predictor.** Left: variance of log output length after removing candidate means, split into the shared problem level, the problem-by-candidate interaction and run-to-run noise (APPS: one draw per problem, so the interaction includes noise); Corr.: mean correlation between two candidates' mean log lengths. Right: cost saved relative to median pricing by oracles that know a problem's shared level, its candidate-specific part, or both (the headroom).

G Three further model families

Three further model families. We added Qwen3-32B, GLM-4.7-Flash and Nemotron-3-Super-120B, all reasoning, on LCB, Omni-MATH and a 2,700-problem subset of MMLU-Pro (billed \$0.40–0.57 per million output tokens; Qwen3-32B's provider stops at 24,575 output tokens, and Nemotron has no valid answer for 27% of Omni-MATH problems after retries), and fitted the same probes for them (test success AUC .72–.87, log-length R^2 .23–.81). Averaged over ten training sets, with all eight candidates our router saves 28.8%, 25.6% and 23.0% relative to median pricing, beats length from predicted difficulty (15.5 [6.8, 23.2]) and from predicted success (6.7 [0.3, 12.2]) on MMLU-Pro, and tie both on LCB and Omni-MATH, as with five candidates. In the full pool the new models are never needed (removing any one changes the saving by under 0.2 points on one training set): deepseek-v4-flash is cheaper than each of them at a higher accuracy. We therefore also route in a pool of both gpt-oss-20b efforts and the three new models, without deepseek-v4-flash or gpt-oss-120b. There our router saves 29.5% [17.3, 37.2], 12.4% [−0.2, 21.7] and 22.7% [9.9, 33.6] relative to median pricing on LCB, Omni-MATH and MMLU-Pro, beats length from predicted difficulty on LCB (7.9 [0.3, 14.3]) and MMLU-Pro (12.5 [0.9, 23.6]) and ties length from predicted success; with all

candidates except deepseek-v4-flash they save 25.3%, 11.4% and 22.4%. Without deepseek-v4-flash no single candidate is cheap and accurate: the most accurate one, Nemotron-3-Super-120B (85.2%, 75.2% and 82.4%), costs \$6.6, \$7.5 and \$2.1 per thousand calls against \$0.6, \$0.9 and \$0.2 for gpt-oss-20b at medium effort; our router's most accurate policy ends 1.6–3.2 points below it. With gpt-oss-120b in the pool, the router reaches Nemotron's accuracy on LCB at 64.8% [42.3, 83.8] lower cost. Onboarding is tested in the pool of cheap candidates (ten training sets, 20 random choices of the k anchors in each). There only Nemotron matters (without it the saving falls from 29.5% to 21.1% on LCB, 12.4% to 10.5% on Omni-MATH and 22.7% to 8.1% on MMLU-Pro). Onboarded from 10 problems, our router saves 15.4%, 11.0% and 15.5%, while ZeroRouter's onboarding saves −35.1%, −30.2% and −6.6% and onboarding at median length and base rate −77.3%, −49.0% and −26.7%. Ours leads ZeroRouter's onboarding by 50.4, 41.2 and 22.2 points on average, with intervals excluding zero in 6, 2 and 1 of the ten training sets; at k=50 by 32.4, 85.3 and 16.1 points, in 10, 8 and 6. A badly onboarded model can cost more than it adds: from 10 examples, onboarding at median length and base rate leaves the router spending more than median pricing for seven of the nine model–dataset pairs, ZeroRouter's onboarding for six, ours for none. Finally (one training set), the new models' probes trained on Omni-MATH and used on MMLU-Pro keep the full saving (28.5% against 28.6% in-domain).

H Generalisation details

	LCB	Omni-MATH	MMLU-Pro	SuperGPQA	BBEH
Ours, k=10	19.5	22.7	19.2	26.7	9.7
ZeroRouter onboarding, k=10	7.3	17.4	9.8	13.7	3.4
CARROT-style kNN, k=10	5.2	11.4	9.5	13.8	8.3
Median length + base rate, k=10	−13.6	7.6	2.1	9.0	1.8
Ours − ZeroRouter, k=10	12.2 [−5.9, 26.7]	5.2 [−7.9, 18.2]	9.4 [−0.9, 19.3]	13.0 [−1.8, 25.9]	6.3 [−11.5, 29.1]
Ours − ZeroRouter, k=50	9.6 [3.0, 18.7]	5.1 [−5.4, 17.7]	8.1 [−0.5, 16.5]	12.8 [4.8, 20.8]	4.8 [−1.5, 10.5]
All candidates on full probes	27.8	31.5	23.4	33.0	23.5

Table 13: **Onboarding a new model from k examples.** Cost saved (%) relative to the full pool at median pricing, mean over three held-out models (gpt-oss-20b with both efforts, gpt-oss-120b with both efforts, deepseek-v4-flash) and four random draws of the k anchors per training set (the same anchors in every arm). Differences in points, with 95% intervals.

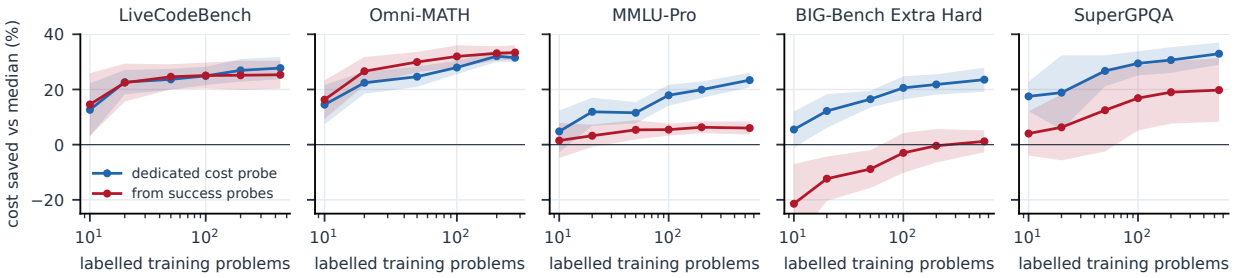


Figure 6: **Length labels needed.** Cost saved relative to median pricing when cost is fitted on n labelled training problems: our cost probe (blue) versus length from predicted success (red), which needs length labels only to map success logits to length. Two random subsets per training set; bands: ±1 s.d. over training sets.

Onboarding anchors. Over the ten training sets of Table 13, ZeroRouter's Fisher-information anchors help both methods about equally; with the same anchors ours ties on LCB and Omni-MATH and leads by 3.5–12.9 points on the other three datasets at k=10.

Combining source and target labels. The remaining results in this appendix are on one training set. Combining both sources of labels is better than either: with ten labelled target problems added to the source

data, the cost probe saves 27.9% on MMLU-Pro and 13.1% on AIME (with the target's success probes), against 8.1% and 4.4% from the ten problems alone, and 24.1% against 14.7% on BIG-Bench Extra Hard with twenty.

Transferring length from predicted success. Length from predicted success transfers unevenly. With the target's own success probes and a map trained on the other datasets, it keeps its saving where the target's kind of difficulty resembles its sources' (APPS 17.1%, AIME 12.3%, CodeContests 9.3%) and collapses elsewhere: from 34.4% to −6.2% on LCB, 19.6% to −21.9% on SuperGPQA, 18.0% to 6.5% on Omni-MATH and 5.5% to −20.5% on BIG-Bench Extra Hard. Under the same conditions the cost probe saves 30.2%, 17.7%, 15.7% and 16.1% on those four, and ZeroRouter's difficulty-bin pricing transfers as unevenly as length from predicted success.

Which estimators lose least. Holding the target's success probes fixed, we bootstrap over test problems the drop from in-domain to transferred pricing for each estimator, and its difference from our drop. Ours loses significantly less than length from predicted success on LCB (37.1 points [26.9, 46.0]), SuperGPQA (31.8 [16.3, 49.9]) and BIG-Bench Extra Hard (13.4 [1.4, 23.9]), and less than ZeroRouter's bin pricing on LCB (21.0 [11.7, 28.5]) and MMLU-Pro (4.0 [1.1, 8.2]); on the other datasets the drops are within noise of each other, and on no dataset does ours lose significantly more. Our own drop is significant only on the two heterogeneous datasets (9.8 and 12.5 points).

Why the cost probe transfers. Training one probe on the target together with the other datasets matches the target-only probe within 4.4 points on every dataset, with no difference significant, so a single probe can serve all of them. Part of what transfers is shared structure: 24–40% of each candidate's variance in log length lies between benchmarks, and the candidates agree on which benchmarks are long (mean correlation of benchmark means across candidates .84). Length R^2 under transfer is close to in-domain on LCB, APPS, AIME, Omni-MATH and SuperGPQA but falls sharply on MMLU-Pro (.02–.46 against .29–.63), where routing transfers fully nonetheless: routing needs the ordering of problems within a candidate and the contrast between candidates, not calibrated absolute lengths. What does not transfer is the map from difficulty to length. Matching the problems of two benchmarks by oracle difficulty (measured solve rate per candidate, five bins), their lengths still differ by 1.14 log units on average (×3.1 over 470 cells). Our cost probe, trained on neither benchmark, predicts these gaps (correlation .95, 90% of the squared gap), length from predicted success 30%, and any price from oracle difficulty alone none, by construction. The curve from solve rate to length also differs in shape: it is flat on BigCodeBench, and without BigCodeBench among LCB's sources a linear map from the mean success logit to length transfers to LCB (12.5% to 32.5%; in-domain 34.7%). The map from difficulty to length is benchmark-specific, while the map from the prefill to length predicts the gaps between benchmarks at matched difficulty.

I Candidate subsets and length spread

Candidate subsets. We refit the router on every subset of the five candidates that leaves out one candidate, and on every pair (Table 14; ten training sets). Choosing effort within one model is worth little: with only gpt-oss-20b's two efforts ours saves 20.2% (LCB), 16.8% (Omni-MATH) and 9.0% (MMLU-Pro) relative to median pricing, with only gpt-oss-120b's 0.2%, 7.0% and 6.0%. Leaving out the cheapest candidate, gpt-oss-20b at low effort, removes most of the saving (LCB 27.8% to 6.5%, Omni-MATH 31.5% to 16.2%, AIME 14.9% to −0.2%, SuperGPQA 33.0% to 0.7%, BIG-Bench Extra Hard 23.5% to 3.8%); leaving out deepseek-v4-flash removes less (to 22.4%, 15.2%, 7.0%, 15.4% and 26.2%; MMLU-Pro 23.4% to 17.0%), and leaving out either gpt-oss-120b effort or gpt-oss-20b at medium effort changes the saving by at most 2.1 points on LCB, Omni-MATH, MMLU-Pro, AIME, SuperGPQA and BIG-Bench Extra Hard.[1] Among pairs, the saving lives in the cheap candidate paired with a strong candidate whose length varies widely: gpt-oss-20b at low effort with deepseek-v4-flash alone saves 33.8% on Omni-MATH, more than the full pool, and 27.0% on MMLU-Pro; with gpt-oss-120b at high effort it saves 32.1% on LCB. deepseek-v4-flash has

[1]Without gpt-oss-20b at low effort on MMLU-Pro, deepseek-v4-flash has the lowest median cost per call, median pricing sends almost every question to it, and its frontier no longer overlaps ours, so no saving is defined.

the widest length spread of any candidate on all nine datasets (s.d. of log length 1.72 on LCB and Omni-MATH, against at most 1.40 and 1.47 for the other candidates), and on seven datasets its spread of cost per call exceeds the spread of prices between candidates; our probe reads its length no better than the other candidates' (Table 15), and on the knowledge datasets worse (Table 11). Per-query cost decides, query by query, whether a strong candidate whose cost varies widely is worth calling instead of a cheap one.

		Without					Pair: gpt-oss-20b low and	
Dataset	All five	20b low	20b med.	deepseek	120b med.	120b high	deepseek	120b high
LiveCodeBench	**27.8**	6.5	**28.4**	**22.4**	**27.8**	**28.3**	**29.0**	**32.1**
Omni-MATH	**31.5**	**16.2**	**32.6**	**15.2**	**31.8**	**32.0**	**33.8**	**22.4**
MMLU-Pro	**23.4**	–	**25.5**	**17.0**	**24.0**	**23.2**	**27.0**	**25.9**
AIME	**14.9**	−0.2	**14.9**	**7.0**	**15.0**	**14.9**	**15.0**	**10.7**
APPS	**16.8**	1.3	**15.0**	**11.5**	11.6	**19.3**	16.3	9.1
BigCodeBench	0.4	−45.0	1.8	3.8	0.8	1.8	2.9	6.2
CodeContests	**9.4**	4.3	**10.1**	**7.1**	9.0	**10.2**	**10.7**	11.2
SuperGPQA	**33.0**	0.7	**33.4**	15.4	**34.0**	**32.8**	**34.6**	24.1
BIG-Bench Extra Hard	**23.5**	3.8	**21.9**	**26.2**	**22.9**	**24.3**	**20.9**	**33.5**

Table 14: **Which candidates the saving needs.** Cost saved (%) by our cost probe relative to median pricing, with the router refitted on a subset of the five candidates; bold: 95% interval excludes zero. –: the frontiers do not overlap (MMLU-Pro without gpt-oss-20b at low effort, where deepseek-v4-flash has the lowest median cost per call).

	s.d. of log output length					Price spread	R^2 beyond predicted success	
Dataset	20b low	20b med.	deepseek	120b med.	120b high	between candidates	deepseek	other candidates
LiveCodeBench	0.90	1.34	**1.72**	1.18	1.40	1.07	0.10	0.09
Omni-MATH	0.88	1.34	**1.72**	1.14	1.47	0.99	0.07	0.14
MMLU-Pro	0.81	0.90	**1.29**	0.75	1.05	0.92	0.44	0.37
AIME	0.51	0.92	**1.06**	0.78	0.88	0.79	0.03	0.07
APPS	0.87	1.31	**1.71**	1.13	1.41	1.14	0.07	0.04
BigCodeBench	0.33	0.37	**0.60**	0.34	0.52	0.80	0.33	0.34
CodeContests	0.65	1.06	**1.29**	0.92	1.04	1.11	0.09	0.04
SuperGPQA	0.73	0.78	**1.23**	0.72	0.96	1.10	0.17	0.16
BIG-Bench Extra Hard	1.15	1.23	**1.26**	1.15	1.02	1.25	0.45	0.48

Table 15: **Length spread per candidate** (one training set). s.d. of log output length over problems (bold: widest on the dataset); price spread: s.d. over candidates of log mean cost per call; R^2 beyond predicted success: our probe's test log-length R^2 minus that of length from predicted success (last column: mean over the other four candidates).

J Representation and prefill size

Representation, probe and prefill size. To compare representations at equal size, Table 16 reads cost from Qwen3-Embedding-4B (Zhang et al., 2025), a text-embedding model with the prefill's parameter count, with three probes (our ridge recipe, a MixLLM-style ensemble and CARROT-style k-nearest neighbours) and our success predictions, and builds a fully text-based router that also predicts success from the embedding with our logistic recipe. On LCB, Omni-MATH, MMLU-Pro and AIME our probe saves 9.7–23.1% relative to every embedding cost probe and 6.8–18.5% relative to the text router, with intervals excluding zero (ahead in every training set against the cost probes, and in 8–10 of 10 against the text router). On the other five datasets it is ahead of every embedding cost probe on average and never significantly behind any arm (the text router is 4.7 points cheaper on CodeContests, within noise). The embedding's success predictions are weaker on every dataset (AUC .73–.75 against .83 on LCB and Omni-MATH). We do not charge the embedding's own encoder pass, which would only favour our cost probe. On PCA-reduced 4B prefill features the three probes are within about 7 points of one another (one training set), so the gain comes from the representation rather than the regressor. Table 17 varies the prefill model while keeping prompts, layers and

	Ours vs. embedding cost probe			Ours vs.	Success AUC	
Dataset	Ridge	MixLLM	kNN	text router	text	ours
LiveCodeBench	**16.9**	**19.5**	**23.1**	**12.0**	.75	.83
Omni-MATH	**15.8**	**18.8**	**19.6**	**18.5**	.73	.83
MMLU-Pro	**15.8**	**13.3**	**12.0**	**12.6**	.64	.72
AIME	**9.7**	**10.1**	**12.2**	**6.8**	.65	.78
APPS	6.5	11.7	9.0	5.0	.75	.84
BigCodeBench	1.2	2.0	0.8	4.3	.61	.64
CodeContests	1.3	3.0	5.7	−4.7	.65	.74
SuperGPQA	7.0	10.3	**13.3**	7.0	.54	.57
BIG-Bench Extra Hard	5.4	4.1	4.6	8.4	.76	.79

Table 16: **A text-embedding model of the prefill's size.** Cost saved (%) by our router relative to routers that read cost from Qwen3-Embedding-4B (our ridge recipe, a MixLLM-style ensemble, CARROT-style kNN; all with our success probes) and to a fully text-based router that also predicts success from the embedding. Bold: 95% interval excludes zero. Success AUC: mean over candidates on the test set.

both probes fixed (ten training sets). Every Qwen3 size from 0.6B to 4B saves 17.0–20.6% on MMLU-Pro and 27.9–29.0% on Omni-MATH against median pricing with its own success probes, and length R^2 rises with size (MMLU-Pro .33, .40, .44; Omni-MATH .61, .65, .66). The result is not specific to Qwen: prefills from three other families (Phi-4-mini (Microsoft, 2025), Granite-3.3-2B, SmolLM2-1.7B (Allal et al., 2025)) save 16.9–21.4% and 25.1–28.2%. None beats our prefill on average: ours saves 2.0–8.2 points relative to each of them, with pooled intervals excluding zero against Qwen3-0.6B, Granite and SmolLM2 on MMLU-Pro and including zero otherwise. Even the same-size Qwen3-4B hybrid release trails by 3.4 and 3.6 points on average (intervals including zero). Any small prefill works; size and release change the saving by a few points.

Fine-tuning. A router could instead train a small model end to end on the length labels, as CARROT does with RoBERTa (Somerstep et al., 2025). We fully fine-tuned three such models on the same training problems and labels (DistilBERT 66M, jina-embeddings 137M and Qwen3-0.6B; learning rate and epoch chosen on calibration error) and used each as a cost estimator with our success predictions. On one training set, each saves less than our probe on Omni-MATH and MMLU-Pro: ours saves 11.9–25.6% relative to each, with intervals excluding zero. LCB is the exception for the largest model: there ours saves 11.7–14.9% relative to the two smaller encoders, but averaged over ten training sets (each fine-tuned anew) only 6.5% [−1.2, 12.2] relative to the fine-tuned Qwen3-0.6B, ahead with an interval excluding zero in 7 of 10 training sets. With a few hundred labelled problems, fine-tuning does not recover more than a linear probe of the frozen model: the same Qwen3-0.6B, frozen and read linearly, saves 31.9% and 22.0% relative to median pricing on Omni-MATH and MMLU-Pro (one training set), more than its fine-tuned copy (20.0% and 19.3%), despite pairing with its own weaker success probes (length R^2 .62 against .53 on Omni-MATH, .33 against .38 on MMLU-Pro). The length information is already present in a pretrained decoder's prefill, and a linear probe recovers it from activations the router already computes, without training a model per router.

The prefill router's success pipeline. The prefill router (Varshney et al., 2026) selects a layer, a pooling and a PCA dimension per target by cross-validation over the encoder's upper layers, concatenates the selected features and trains an ensemble of shared-trunk MLPs. We reimplemented this pipeline on the same encoder, splits and cost probe, and compared it with our per-candidate logistic probes on fixed layers. On one training set the two are tied on all three test sets: ours saves −0.1% [−7.7, 4.0] on LCB, 1.1% [−1.0, 3.0] on MMLU-Pro and −2.2% [−5.7, 1.9] on Omni-MATH relative to it (success AUC .857 vs. .839, .715 vs. .714 and .829 vs. .842). On the two heterogeneous datasets the comparison splits: ours saves 8.4% [−2.5, 18.4] relative to their pipeline on BIG-Bench Extra Hard, and their pipeline is ahead by 6.0% [0.4, 11.2] on SuperGPQA, where success is barely predictable for either (AUC .58 and .57). Linear probes on a fixed set of layers are enough on four of five sets, and they make the cost probe a drop-in addition: the same features feed both. They are also cheaper to fit: on 16 CPU threads, our success and cost probes for five candidates take about 3 minutes on LCB and MMLU-Pro (nine regularisation values per candidate, chosen on calibration), against about 7.4 minutes for the reimplemented pipeline (48 feature configurations × 5

	Omni-MATH			MMLU-Pro		
Prefill	R^2	vs. median	Ours vs. it	R^2	vs. median	Ours vs. it
Qwen3-0.6B	.61	27.9	7.3 [−0.6, 15.7] (5)	.33	17.0	**8.2 [0.5, 14.0]** (8)
Qwen3-1.7B	.65	29.0	4.1 [−5.5, 13.4] (4)	.40	19.4	5.2 [−1.5, 12.1] (7)
Qwen3-4B (hybrid)	.66	28.1	3.6 [−6.5, 12.3] (4)	.44	20.6	3.4 [−2.5, 7.4] (8)
Phi-4-mini (3.8B)	.64	28.2	7.6 [−2.4, 16.5] (7)	.43	21.4	2.0 [−6.2, 6.8] (5)
Granite-3.3 (2.5B)	.62	25.1	5.6 [−3.3, 14.6] (3)	.38	18.2	**7.1 [2.1, 12.8]** (10)
SmolLM2 (1.7B)	.58	25.2	5.9 [−3.7, 14.0] (6)	.33	16.9	**7.5 [2.1, 11.5]** (9)
Qwen3-4B-2507 (ours)	.70	31.5	–	.49	23.4	–

Table 17: **Other prefills.** Each prefill feeds both probes, refitted with the paper's recipes (Omni-MATH and MMLU-Pro, billed costs). R^2: test log-length R^2, mean over candidates. vs. median: cost saved (%) relative to median pricing with the row's own success predictions. Ours vs. it: cost saved by our router relative to the row's, with 95% intervals (bold: excludes zero) and the number of training sets in which ours is significantly ahead; ours is significantly behind in one training set only (Phi-4-mini, MMLU-Pro).

folds per candidate, then ten networks); neither needs an encoder pass beyond the shared prefill, unlike the MixLLM-style and ZeroRouter estimators, which fit in seconds but read their own text encoder.

K Deployment

Routing near the top. The router does not beat the best single model at that model's accuracy. On every dataset deepseek-v4-flash is the most accurate candidate or within 1.6 points of it, at 17–37% of the cost of gpt-oss-120b at high effort (Table 2). The router's most accurate policy, which sends each query to the candidate with the highest predicted success, ends 0.2–2.7 points below the most accurate candidate on seven datasets (it ties on AIME and is 2.0 points above on BIG-Bench Extra Hard), and where it reaches deepseek-v4-flash's accuracy it spends as much as calling it alone (AIME, SuperGPQA, BIG-Bench Extra Hard) or more (the coding datasets). Close to the top, routing buys little over calling deepseek-v4-flash alone: giving up one point of accuracy saves 5.1% on AIME and 9.4% on SuperGPQA (intervals excluding zero) and nothing significant elsewhere, while giving up five points saves 14.9–40.3% on the six datasets other than APPS, BigCodeBench and CodeContests (LCB 32.6%, Omni-MATH 33.1%, MMLU-Pro 26.9%, BIG-Bench Extra Hard 40.3%), 1.5–1.8× what median pricing and 1.5–3.6× what a random mix of candidates saves at the same accuracy. Below the top it is much cheaper than every other candidate at that candidate's accuracy: on the same six datasets, 45–83% cheaper than gpt-oss-20b at medium effort and, where it reaches its accuracy, 50–92% cheaper than gpt-oss-120b at high effort. The shortfall at the top is not specific to deepseek-v4-flash. In the pool of Appendix G without it or gpt-oss-120b, the most accurate candidate, Nemotron-3-Super-120B, costs \$2.1–7.5 per thousand calls and the router again ends 1.6–3.2 points below it; there our router saves 44.7% [36.6, 52.2] relative to the mix on LCB and 22.7% [1.9, 38.3] on Omni-MATH, against 21.4% and 10.9% with median pricing, and on MMLU-Pro neither beats the mix (ours −7.6% [−39.3, 20.5], median pricing −39.9%).

Routing versus cascades. A cascade calls candidates in sequence and stops when an answer is accepted (Chen et al., 2023; Aggarwal et al., 2024). To bound what any cascade could achieve on our datasets, we give it a perfect judge: every ordered plan of the five candidates is called cheapest first, each attempt is judged by its true outcome at no cost, and the cascade stops at the first correct answer. A learned judge can only do worse. Even against this bound, one-shot routing with our router (one training set) spends 29.7% [19.6, 37.7] less at matched accuracy on LCB and 31.9% [26.4, 37.3] less on Omni-MATH, because a cascade pays for every failed attempt before the one it accepts. On MMLU-Pro the two tie (−0.8% [−6.1, 3.9]): the cheapest candidate solves 62% of questions at about 6% of the most expensive candidate's cost, so failed first attempts are cheap.

Deployable policies. Our main metric compares cost–accuracy frontiers on the test set; a deployed router must fix its trade-off V in advance. For each estimator and each of ten accuracy targets spanning the band every estimator reaches on the calibration split, we choose the cheapest V that reaches the target on calibration and apply it once to the test set. We compare its test cost with median pricing's deployed policies, interpolated at the accuracy it actually reaches (Table 18; ten training sets). Our deployed policies save 19.6–37.9% on LCB, Omni-MATH, MMLU-Pro, AIME, SuperGPQA and BIG-Bench Extra Hard; they cost 1.0–3.3% more than our own test frontier at the accuracy they reach and miss their target by 2.2–5.3 points. They lead the best deployed external estimator on LCB, Omni-MATH and MMLU-Pro with intervals excluding zero in every training set, on AIME in 9 and SuperGPQA in 7 of 10, and tie it on BIG-Bench Extra Hard; they lead length from predicted success on MMLU-Pro and BIG-Bench Extra Hard in every training set and on SuperGPQA in 7. Deployed savings exceed the frontier savings mainly because median pricing deploys less well: its calibration-chosen policies fall 8.6–16.5% short of its own test frontier on MMLU-Pro, AIME, APPS, BigCodeBench, CodeContests and SuperGPQA, against at most 9.4% for ours. On APPS, BigCodeBench and CodeContests our lead is within noise.

A frontier reference. To place the dataset against a frontier model, we called Claude Opus 5.5 (adaptive thinking at high effort, one sample per problem, same prompt and grader) on 100 random LiveCodeBench test problems. It solves 94.0% at $21.9 per thousand calls; it thinks on only 24% of these problems and writes 954 output tokens on average. On the same problems deepseek-v4-flash alone reaches 88.8% at $0.8 per thousand calls, 94% of the frontier model's accuracy at 4% of its cost, and gpt-oss-120b at high effort 89.3% at $3.4; our router's most accurate policy reaches 87.7% at $1.6 (mean over the ten training sets), no better than deepseek-v4-flash alone, as on the full test set. The gap to the frontier model is a property of the pool's strongest open candidate, not of routing; the router's saving lies below that candidate's accuracy.

The router's own cost. The features come from one forward pass of Qwen3-4B over the prompt: 127–1,216 tokens on average per dataset, 19–28 ms median latency on one H100 at batch size one. A router that predicts success from the prefill already pays for this pass, so the cost probe adds nothing to it. Against no routing at all, the pass is not free when the routed models are as cheap as ours (well under a dollar per thousand queries). Self-hosted at an assumed $2 per H100-hour with unoptimised batched inference, it costs 2–10% of our deployed saving on Omni-MATH, AIME, MMLU-Pro and SuperGPQA and about 30% on BIG-Bench Extra Hard and APPS, whose prompts are long; at the input price of the smallest Qwen3 model listed on OpenRouter (14B, $0.12 per million tokens), it costs 6–51% of the saving, and more than the saving on APPS. Its share falls as the routed models get more expensive.

Per-query budgets. A per-query budget is the one setting where knowing when the strongest candidate will run long could beat calling it alone at any accuracy. We cap the cost of every call at X (a call that would exceed it is cut off and fails), with X at the 50th, 70th, 80th, 90th and 95th percentile of deepseek-v4-flash's cost per call on the training and calibration problems, and send each query to the candidate with the highest predicted probability of succeeding within the cap (ten training sets, full test sets). With our probe, that probability is the success probe times the chance that the candidate's length fits, from the predicted length and the training residuals. At the tightest cap the router beats calling deepseek-v4-flash alone on LCB (8.2 points of accuracy [5.8, 10.8]) and MMLU-Pro (7.0 [5.7, 8.3]), ahead in every training set, and the best single candidate by 5.3 and 0.9 points; on Omni-MATH it is 1.8 points ahead [−0.6, 4.3]. At looser caps it ties deepseek-v4-flash alone or falls 1–3 points behind (intervals excluding zero on MMLU-Pro and Omni-MATH at the 90th and 95th percentiles), at a spend between 19% below and 11% above deepseek-v4-flash's. Knowing each problem's realized length would beat deepseek-v4-flash alone by 9.3–15.5 points at the tightest cap and by 3.9–9.7 at the 70th percentile, so there is room under budgets; a probe trained directly on whether a call fits within the cap (a logistic probe per candidate and cap on the same prefill) reads it no better (within 2 points of ours everywhere, with the same AUC for whether deepseek-v4-flash fits, .81–.93). At the 95th percentile even known lengths do not beat deepseek-v4-flash alone, so there, too, the limit is the success probes. Against median pricing under the same caps our probe is ahead by up to 12.6 points (LCB at the 90th percentile, where median pricing sends queries to candidates whose median length fits the cap but whose long outputs do not), and behind significantly at no cap.

Dataset	Ours vs median, deployed	Best external (deployed)	Ours − best external	Ours − cost from success	Ours: frontier loss / target miss
LiveCodeBench	31.3 (7.0)	MixLLM 14.9	16.3 (10)	2.7 (2)	2.4 / 2.6
Omni-MATH	35.0 (2.2)	ZeroRouter 13.9	21.5 (10)	−1.6 (0)	2.0 / 5.3
MMLU-Pro	31.1 (4.8)	kNN 18.1	12.7 (10)	17.3 (10)	1.0 / 2.3
AIME	19.6 (2.1)	ZeroRouter 10.3	9.4 (9)	2.2 (2)	1.3 / 4.6
APPS	23.3 (5.1)	ZeroRouter 15.8	5.2 (1)	−2.4 (0)	9.4 / 3.6
BigCodeBench	7.0 (5.2)	kNN 5.7	2.3 (0)	3.8 (1)	9.2 / 2.2
CodeContests	13.5 (4.4)	kNN 8.4	5.7 (2)	−0.3 (0)	6.3 / 5.6
SuperGPQA	37.9 (7.1)	MixLLM 24.6	14.1 (7)	15.4 (7)	3.3 / 2.5
BIG-Bench Extra Hard	24.0 (4.5)	MixLLM 24.5	−1.0 (0)	28.7 (10)	2.2 / 2.2

Table 18: **Deployable policies.** Each estimator picks on the calibration split the cheapest V reaching each of ten accuracy targets, applied once to the test set. Cost saved (%) relative to median pricing's deployed policies at the accuracy reached, mean over targets (in parentheses: s.d. over training sets for our saving; for differences, the number of training sets in which ours is significantly ahead; ours is significantly behind only in one APPS and one Omni-MATH set, against length from predicted success). Last column: extra cost over our own test frontier at the accuracy reached (%) / mean |test accuracy − target| (points). Billed prices.

L Software engineering

We study software engineering in two regimes: one-shot patch generation, which behaves like the coding datasets, and agents, where the price gaps between models dominate.

One-shot patches. On 1,432 SWE-Smith instances (Yang et al., 2025b), each of the five candidates writes one patch (deepseek-v4-flash pinned as elsewhere), labelled by executing the repository's tests in a sandbox. Pass rates are low and close together (25–37%), but which instances are solvable is predictable from the issue (success AUC .80–.83 per candidate on the 389 test instances where every candidate returned a patch). Output length varies 5–28× between a candidate's 10th- and 90th-percentile instances and is only modestly predictable (log-length R^2 .09–.37). Over ten training sets, per-query cost knowledge is worth 25.4% [9.8, 35.5] over median pricing, and our cost probe saves 7.3% [−4.7, 17.8], a capture of .29. No estimator differs significantly from ours: we save 5.7% [−1.6, 11.9] relative to mean pricing, 3.6% [−7.0, 13.5] relative to the prompt-feature GBM, 5.5% [−3.8, 14.5] relative to MixLLM-style embeddings and 2.2% [−4.8, 14.1] relative to length from predicted success, and ZeroRouter's whole router is within noise of ours (−5.9% [−22.5, 11.6] with the 4B prefill). One-shot patch generation thus behaves like the coding datasets where routing pays least: the headroom is real, but little of it is captured.

Agents: cost is real, varies by task and is stable. Public SWE-rebench trajectories (Badertdinov et al., 2025) give 111 tasks run five times by each of 13 models in one scaffold. Across models, agentic cost is dominated by the model: of the variance of log cost, 74% is between models, 15% between tasks, 6% their interaction and 4% run to run. Within a model, however, per-task cost spans 2.7–9.0× between the 10th and 90th percentile, and it is stable across runs (intraclass correlation .69–.92). Whether knowing it pays depends on the price gaps between the routed models, as in Fig. 7: among all 13 models, whose prices per run span about 190×, per-task cost knowledge is worth 8.5% [−5.0, 17.8] (one-shot routing, success and cost from three runs, scored on the other two); among the seven open-weight models, whose prices span about 10×, it is worth 25.7% [6.3, 40.6], as much as on the reasoning datasets.

Agents: cost is only partly predictable before the run. From the issue text alone, the prefill's cost probe reaches out-of-fold R^2 of about zero on the 111 SWE-rebench tasks, too few to fit it. On 3,387 tasks with about 20 SWE-agent runs each (one model, Llama-3.1-70B (Nebius, 2024); cost proxied by cumulative context and output length), the per-task mean cost is reliable (split-half reliability .91) and the issue text predicts it at R^2 .29, still rising with data (.18 with 100 tasks, .32 with 2,710). This is lower than what the prefill reaches on most one-shot datasets (Table 11). Reading the first ten steps of a cheap model's trajectory

predicts other models' per-task cost at R^2 .16 and the same run's final cost at .28, which suggests exploring cheaply before pricing the expensive candidates.

M Pre-registered predictions

Before collecting two of the datasets, we recorded a forecast of whether the cost probe would pay there, from a cheap screen. The rule for Omni-MATH-500 was fixed before its screen: from one gpt-oss-20b-low draw per problem, if the prefill's cross-validated log-length R^2 was at least .50, predict a saving of at least 10% over median pricing with an interval excluding zero; if at most .35, predict none; in between, make no call. The screen gave .67, and the forecast (a gain, and headroom of at least 15%) was confirmed on the full set at the time, on one training set: 21.9% [8.9, 33.5] at list prices with the pre-registered Thinking prefill, headroom 44.2%. The same rule run as a screen elsewhere predicted a gain on APPS, which was not significant before deepseek-v4-flash was pinned and is at pinned billed prices (16.8% [4.2, 30.4]) and wrong on AIME, where it predicted no gain and our probe saves 14.9% [10.8, 20.1]: a single draw of the cheapest candidate understates how predictable lengths are. A second screen chose SuperGPQA and BIG-Bench Extra Hard as datasets where the cost probe should beat length from predicted success: on one gpt-oss-20b-low draw per problem, length predicted directly from the prefill beat length predicted from a success probe by a margin as large as MMLU-Pro's (R^2 gap .61 and .40, against .66 on MMLU-Pro and .10 on Omni-MATH); it did on BIG-Bench Extra Hard (22.6 [12.3, 31.9]) and overstated the gap on SuperGPQA (15.3 [−0.6, 32.3]; ahead in 7 of 10 training sets).

N Random re-splits and price gaps

Random re-splits. Table 19 repeats the main comparison on ten random re-splits that also reshuffle the test problems.

Dataset	Headroom	Ours vs median	Ours − best external	Splits ahead of all external	Ours − cost from success	Splits ahead / behind
LiveCodeBench	45.9 ± 3.0	34.1 ± 3.5	12.9 ± 2.9	10	2.2 ± 2.6	4 / 0
Omni-MATH	41.7 ± 2.5	19.0 ± 3.3	12.0 ± 3.8	9	−1.4 ± 2.6	1 / 2
MMLU-Pro	58.6 ± 1.0	25.8 ± 2.3	16.3 ± 2.6	10	12.8 ± 6.1	10 / 0
AIME	28.5 ± 2.7	15.3 ± 2.6	9.5 ± 2.6	10	2.5 ± 2.4	3 / 0
APPS	39.5 ± 6.0	15.6 ± 4.9	3.9 ± 2.8	0	−1.5 ± 2.4	0 / 0
BigCodeBench	11.0 ± 6.2	3.6 ± 2.3	−1.8 ± 4.7	0	4.2 ± 2.7	1 / 0
CodeContests	34.8 ± 2.0	8.7 ± 4.5	2.1 ± 4.0	0	−0.1 ± 3.3	0 / 0
SuperGPQA	59.0 ± 2.8	32.1 ± 5.8	5.8 ± 5.0	3	10.4 ± 7.6	5 / 0
BIG-Bench Extra Hard	37.6 ± 3.5	24.8 ± 5.7	1.0 ± 1.1	1	20.6 ± 5.3	10 / 0

Table 19: **Ten random re-splits per dataset**, which, unlike the training sets, also reshuffle the test problems (original sizes; every probe and estimator refitted, configurations chosen on calibration). Mean ± s.d. over splits of cost saved (%) relative to median pricing, and of our saving relative to the best external estimator in each split and to length from predicted success. "Splits ahead": splits in which ours is significantly ahead of every external estimator, and (last column) ahead of / behind length from predicted success. No external estimator is significantly cheaper than ours in any split. Values differ from Table 3; the conclusions do not.

Price gaps. Figure 7 rescales every candidate's per-token price toward or away from the others' while keeping realized lengths fixed (one training set). On LCB and Omni-MATH our saving grows with the price gap and saturates near the billed gap (3.1× between the most and least expensive candidate); on MMLU-Pro it is largest at flat prices. Beyond the billed gap the headroom shrinks: at a 29× spread it is 7–49% across datasets, at 87× 2–42%, and our saving falls with it (1–25% at 87×).

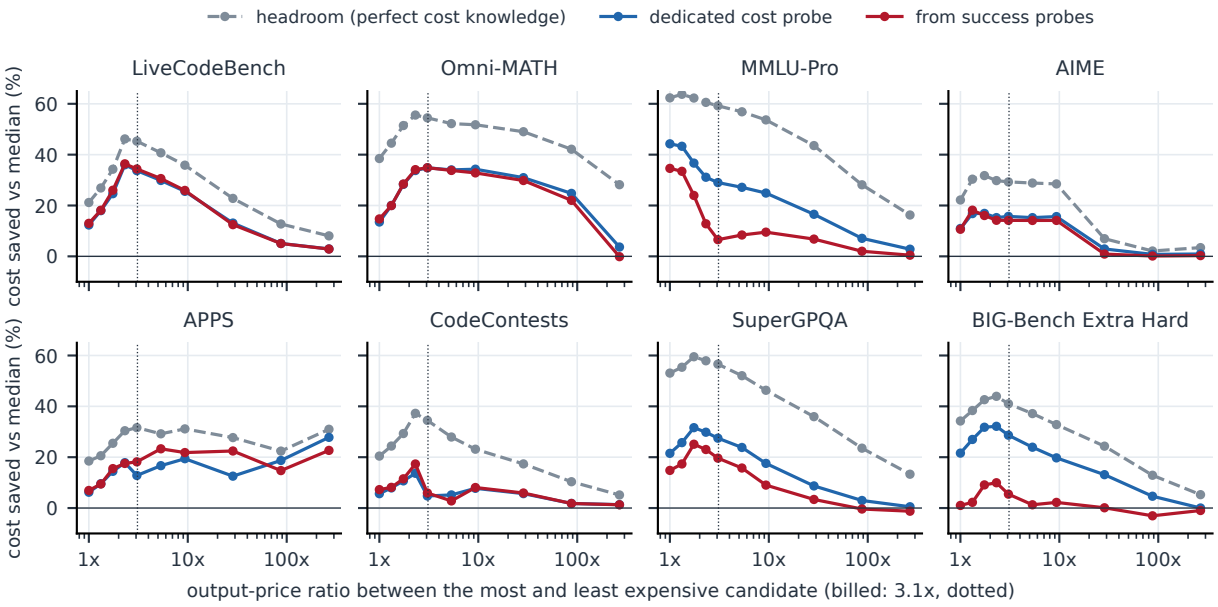


Figure 7: **Price gaps.** Cost saved relative to median pricing as every candidate's per-token price is moved toward or away from the candidates' geometric mean (r_m^α, rescaled; α from 0, flat, to 5), with realized lengths fixed and costs recomputed from token counts. Dashed: headroom; blue: our cost probe; red: length from predicted success; dotted: the billed spread (3.1×). BigCodeBench is omitted (its matched-accuracy band degenerates at some spreads).